VIETNAM-KOREA UNIVERSITY OF INFORMATION
TECHNOLOGY AND COMMUNICATION TECHNOLOGY

Computer Science Faculty

FOUNDATIONAL PROJECT 4

STOOM – STUDY TOGETHER PLATFORM

Student: **Nguyen Thanh Vinh - 23IT313**

Class: **23GIT**

Instructor: **Ho Van Phi, Dr.Sc.**

Da Nang, January 2026

VIETNAM-KOREA UNIVERSITY OF INFORMATION TECHNOLOGY AND COMMUNICATION TECHNOLOGY

Computer Science Faculty

FOUNDATIONAL PROJECT 4

STOOM – STUDY TOGETHER PLATFORM

Student: **Nguyen Thanh Vinh - 23IT313**

Class: **23GIT**

Instructor: **Ho Van Phi, Dr.Sc.**

Da Nang, January 2026

INSTRUCTOR'S COMMENT

……

……

……

……

……

……

……

……

……

……

……

……

……

……

ACKNOWLEDGEMENT

I am extremely grateful to have had such an amazing supervisor to support and guide us throughout our project. Your expertise, encouragement, and patience have been invaluable to me, and I am truly honored to have had the opportunity to work under your supervision.

I cannot express enough how much we appreciate the time and effort you have dedicated to helping us achieve our goals. Thank you for your unwavering support, encouragement, and mentorship throughout this journey. I take great pride in having had the opportunity to work alongside you, and I will always hold a deep appreciation for your exceptional dedication and hard work.

Student,

Nguyen Thanh Vinh

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

WORD	MEANING
AI	Artificial Intelligence
API	Application Programming Interface
CRDT	Conflict-free Replicated Data Type
CSR	Client-Side Rendering
DOM	Document Object Model
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
NoSQL	Not Only SQL
ORM	Object-Relational Mapping
P2P	Peer-to-Peer
RSC	React Server Components
SDLC	Software Development Life Cycle
SFU	Selective Forwarding Unit
SQL	Structured Query Language
SSR	Server-Side Rendering
STT	Speech-to-Text
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
WebRTC	Web Real-Time Communication

INTRODUCTION

1. Reason for choosing this topic

Current Problem:

In the era of digital transformation, online learning has become an integral part of education. However, many current platforms primarily focus on audio-visual transmission (Video Conferencing) while lacking specialized interactive tools for group study. Learners often have to use multiple fragmented applications simultaneously—one for video calls (like Zoom, Google Meet) and another for drawing or note-taking (like Miro, Notion)—leading to distraction and reduced learning efficiency. Furthermore, capturing and synthesizing knowledge after each study session is often neglected or done manually, which is time-consuming and prone to missing important information.

Stoom's Solution:

The **"Stoom – Study Together Platform"** is developed to offer a comprehensive, "All-in-one" solution that maximizes support for the Peer-to-Peer (P2P) study model. Stoom not only connects video and audio via WebRTC but also deeply integrates real-time collaboration tools such as an Interactive Whiteboard and Collaborative Notes.

Significance/Necessity:

Building a friendly, stable, and highly interactive online learning environment is an urgent need to improve the quality of digital education. This project serves as a valuable opportunity to apply the most modern web programming technologies (Next.js 16, TypeScript, WebRTC) and Artificial Intelligence (AI) models to solve practical problems. The outcome of this topic aims to create a complete product with an intuitive interface, accessible to students and lecturers, while ensuring data safety and rapid retrieval capabilities.

2. Implementation plan

Main project objectives:

- Analyze requirements and user behaviors in an online group study environment to design a friendly UI/UX.
- Research and select appropriate technologies: Next.js for Frontend/Backend, WebRTC for streaming, and libraries supporting collaboration (tldraw, Y.js).
- Build a real-time synchronized Whiteboard and Collaborative Notes system.
- Integrate AI APIs (Speech-to-Text, LLM) to construct the AI-Insight function.
- Test the system's stability in a real network environment and evaluate the effectiveness of learning support features.
- Finalize a detailed report on the design and implementation process.

Implementation Plan:

Time	Nội dung thực hiện
18/10 - 22/10	Phân tích yêu cầu hệ thống, xác định chức năng chính
23/10 - 31/10	Thiết kế UI/UX và sơ đồ kiến trúc hệ thống
01/11 - 07/11	Thiết kế cơ sở dữ liệu và mô hình kết nối WebRTC
08/11 - 13/11	Phát triển back-end và tích hợp hệ thống lưu trữ
14/11 - 21/11	Tích hợp WebRTC và Whiteboard thời gian thực
22/11 - 26/11	Phát triển Speech-to-Text, Collaborative Notes, AI-Insight
27/11 - 29/11	Kiểm thử, tối ưu hiệu suất và sửa lỗi, viết báo cáo
30/11 – 1/12	Hoàn thiện và nộp báo cáo đồ án

3. Report structure:

INTRODUCTION

CHAPTER 1: THEORETICAL BASIS

CHAPTER2: SYSTEM ANALYSIC AND DESIGN

CHAPTER 3: IMPLEMENTATION

CONCLUSION

Chapter 1. THEORECTICAL BASIC

1.1 Next.js Framework & App Router Architecture

Next.js 16 is utilized as the core full-stack framework for the application. It employs the **App Router** architecture, which supports **Hybrid Rendering**. This allows the application to leverage **Server-Side Rendering (SSR)** for initial load performance and SEO, while using **Client-Side Rendering (CSR)** for interactivity. Crucially, **React Server Components (RSC)** are used to render components on the server, significantly reducing the client-side JavaScript bundle size and improving performance on lower-end devices.

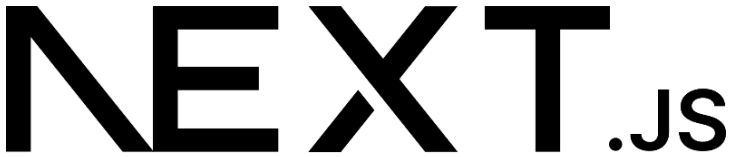

Figure 1-1. NextJS

1.2 Real-Time Communication (WebRTC & LiveKit)

WebRTC (Web Real-Time Communication) is the underlying standard enabling browser-to-browser streaming of audio, video, and data. To address the scalability limitations of traditional Peer-to-Peer (Mesh) networks, Stoom integrates **LiveKit**, an open-source infrastructure based on the **Selective Forwarding Unit (SFU)** architecture. Instead of establishing connections between every participant, each client sends a single stream to the SFU server, which forwards it to others, optimizing bandwidth and CPU usage for group study sessions.

Figure 1-2. WebRTC

Figure 1-3. LiveKit

1.3 Collaborative Synchronization (CRDTs & Y.js)

To enable real-time collaboration on the Whiteboard and Notes without data conflicts, the system relies on **Conflict-free Replicated Data Types (CRDTs)**. The project implements **Y.js**, a high-performance CRDT library. Y.js manages the shared state of the **tldraw** canvas and **Tiptap** editor, ensuring that concurrent updates from multiple users are merged consistently and automatically, even in unstable network conditions.

Figure 1-4.Y.js

1.4 Database & ORM (MongoDB & Prisma)

MongoDB, a document-oriented NoSQL database, is selected for its flexibility in storing unstructured and complex data structures, such as whiteboard vector snapshots and chat logs.

Figure 1-5. MongoDB

Prisma ORM serves as the data access layer. It provides type-safety by automatically generating TypeScript types based on the database schema. This prevents type-related runtime errors and simplifies query logic within the Next.js backend.

Figure 1-6. Prisma ORM

1.5 Authentication & Identity Management

Clerk is employed for secure and complete identity management, replacing custom authentication flows. It handles session management, multi-factor authentication, and OAuth integrations (Google/GitHub). Clerk integrates directly with **Next.js Middleware** to enforce route protection, ensuring only authenticated users can access the dashboard and meeting rooms.

Figure 1-7. Clerk

Chapter 2. SYSTEM ANALYSIS AND DESIGN

2.1 System Requirements

2.1.1 Actors

The system employs an inheritance-based actor model (Generalization/ Specialization) to define user roles and system entities. The base actor is the User, which represents any authenticated individual accessing the platform through Clerk authentication. Specialized actors inherit from this base class, each with distinct permissions and responsibilities.

Base Actor: User

The **User** actor is the fundamental entity in the system, representing any authenticated individual. All users must authenticate through Clerk before accessing protected features. The User actor has the following base attributes:

- Authentication status (managed by Clerk)
- User ID (Clerk user ID)
- Email address
- Profile information (name, avatar)

Specialized Actor: Room Host

The **Room Host** is a specialized User who creates a meeting room. This actor inherits all capabilities of the base User class and additionally possesses administrative privileges within the room context. The Room Host has the following specialized capabilities:

- Create new meeting rooms with custom settings (title, password, media preferences, whiteboard restriction)
- Control room settings during an active session
- Manage collaboration permissions (whiteboard, notes access control)
- Grant or revoke whiteboard/notes editing access to specific participants
- Kick (remove) participants from the meeting
- Grant or revoke co-host privileges to participants
- End the meeting session
- Save whiteboard snapshots and notes to the database

Specialized Actor: Co-Host

The **Co-Host** is a specialized User who has been granted elevated privileges by the Room Host. This actor inherits Participant capabilities and additionally has management rights. The Co-Host can:

- Manage collaboration permissions (whiteboard, notes access control)

- Grant or revoke whiteboard/notes editing access to specific participants
- Kick (remove) participants from the meeting
- End the meeting session
- Lower raised hands of participants

Specialized Actor: Participant

The **Participant** is a specialized User who joins an existing meeting room. This actor inherits base User capabilities but has limited administrative rights. The Participant can:

- Join rooms via room code or direct link
- Participate in real-time collaboration (video, audio, whiteboard, notes)
- Send chat messages
- Use whiteboard and notes based on permission settings
- Raise hand to get attention from host/co-host
- Leave the room at any time

2.1.2 Functional Requirements

Module 1: Authentication and User Management

- **REQ-1.1:** The system shall authenticate users through Clerk authentication service, supporting email/password and social login methods (Google, GitHub).
- **REQ-1.2:** The system shall protect all routes under (dashboard) and (room) route groups using Clerk middleware, redirecting unauthenticated users to /sign-in.
- **REQ-1.3:** The system shall provide sign-in and sign-up pages (/sign-in, /sign-up) with centered card layout, gradient background, and custom violet-themed styling matching the brand identity.
- **REQ-1.4:** Upon successful authentication, the system shall redirect users to /dashboard automatically.
- **REQ-1.5:** The system shall display a UserButton component in the dashboard header, allowing users to access their profile and account settings managed by Clerk.

Module 2: Room Management and Navigation

- **REQ-2.1:** The system shall provide a "Join with Code" dialog on the dashboard, allowing users to enter a room code and navigate to /room/[roomCode].
- **REQ-2.2:** The system shall provide a "New Meeting" dialog that allows Room Hosts to:
  - Set a meeting name (required field)
  - Optionally set a room password for access control
  - Optionally restrict whiteboard access (only host can edit by default)
  - Generate a unique 6-character room code via /api/room/create
  - Persist room to MongoDB with status WAITING and initial permission settings
  - Navigate to the room after creation
- **REQ-2.3**: The system shall validate room access via /api/room/validate endpoint:
  - Check room existence (return 404 if not found)
  - Check room status (return 410 if ENDED or inactive)
  - Perform stale room cleanup (mark ACTIVE rooms as ENDED if no LiveKit room exists for 5+ minutes)
  - Return room info including: code, name, isHost, hasPassword
- **REQ-2.4:** The system shall provide a pre-join screen (/room/[roomId]) before entering the main room:
  - Display video preview with camera feed (or user avatar if camera off)
  - Show user's Clerk profile image when camera is disabled
  - Toggle microphone and camera before joining
  - Display password input field if room has password (hidden for host)
  - Show room name and navigation back to dashboard
- **REQ-2.5**: The system shall manage room lifecycle through LiveKit webhooks (/api/livekit/webhook):
  - room_started: Update room status to ACTIVE, set startedAt timestamp
  - room_finished: Update room status to ENDED, set endedAt timestamp, mark all participants as left
  - participant_joined: Create/update RoomParticipant record with role (HOST or PARTICIPANT)
  - participant_left: Update RoomParticipant leftAt timestamp
- **REQ-2.6**: The system shall allow Room Hosts to end meetings via "End Meeting" button:
  - Display confirmation dialog before ending

- Broadcast ROOM_ENDED message to all participants via LiveKit DataChannel
- Call /api/room/end to delete LiveKit room and update database
- Show "Meeting Ended" overlay to all participants with countdown and redirect

- **REQ-2.7:** The system shall provide a Hand Raise feature for participants:
  - Participants can raise/lower their hand via a button in the meeting controls
  - Raised hands are displayed with a visual indicator (hand icon) on participant cards
  - Host and co-hosts see a hand raise queue showing participants in order of when they raised their hand
  - Host and co-hosts can lower individual participant's hands or lower all hands at once
  - Hand raise state is synchronized in real-time via LiveKit DataChannel
  - Screen reader announcements are provided for accessibility
- **REQ-2.8:** The system shall allow Room Hosts and Co-Hosts to kick (remove) participants:
  - Display a kick button on participant cards (visible only to host/co-hosts)
  - Show confirmation dialog before kicking a participant
  - Send PARTICIPANT_KICKED message to the kicked participant via LiveKit DataChannel
  - Remove participant from LiveKit room via roomService.removeParticipant()
  - Display "You have been removed from the meeting" overlay to the kicked participant
  - Host cannot be kicked from the meeting
- **REQ-2.9:** The system shall allow Room Hosts to manage Co-Host privileges:
  - Host can grant co-host role to any participant via the Collaboration Settings modal
  - Host can revoke co-host role from any co-host
  - Co-host status is persisted in the database (RoomParticipant.role = "CO_HOST")
  - Co-host changes are broadcast to all participants via LiveKit DataChannel
  - Co-hosts have elevated permissions: can manage whiteboard/notes access, kick participants, end meeting, lower hands
- **REQ-2.10:** The system shall support room password management during active sessions:
  - Host can set, change, or remove room password via the Collaboration Settings modal

- Password validation is performed via /api/livekit/token endpoint during join
- Host is exempt from password validation when joining their own room
- Password must be at least 4 characters when set
- Password is stored securely using bcrypt hashing in the database
- API endpoints: GET /api/room/[roomId]/password (check if password exists), PUT /api/room/[roomId]/password (set/update/remove password)

Module 3: Real-time Collaboration

- **REQ-3.1**: The system shall connect to LiveKit server for real-time video/audio communication:
  - Generate LiveKit access tokens via /api/livekit/token with user identity and metadata (imageUrl)
  - Use @livekit/components-react library for room connection and media handling
  - Sync user data to MongoDB during token generation (upsert clerkId, name, email, imageUrl)
- **REQ-3.2:** The system shall implement a multi-panel layout in the room interface:
  - Participants Sidebar: Displays room code, participant count, and list with avatars
  - Main Stage: Displays video feeds grid, screen share, and/or whiteboard
  - Chat/Notes Panel: Tabs for Chat and Personal Notes, can be toggled
- **REQ-3.3:** The system shall use react-resizable-panels library to enable users to resize panels by dragging resize handles, with hover effects showing violet color on handles.
- **REQ-3.4:** The Participants Sidebar shall:
  - Display room code with copy-to-clipboard functionality
  - Show participant count badge
  - Display each participant with: Clerk profile image (or initial avatar), name, "(You)" indicator for local participant
  - Show mic/video status icons (green for enabled, red/gray for disabled)
  - Highlight active speaker with violet border and "Speaking" label
  - Display hand raise indicator (hand icon) for participants with raised hands
  - Show role badges (Host, Co-Host) on participant cards

- Display kick button on participant cards for host/co-hosts (with confirmation dialog)
- Show hand raise queue panel for host/co-hosts with "Lower Hand" and "Lower All" actions
- Support collapse/expand functionality

- **REQ-3.5:** The Video Feeds View shall:
  - Display all participants' video tracks in a responsive grid layout
  - Show participant name overlay on each video tile
  - Display avatar placeholder when video is disabled
  - Highlight active speaker with visual indicator
- **REQ-3.6:** The Screen Share View shall:
  - Display screen share track when a participant is sharing
  - Show sharer's name in overlay
  - Allow any participant to share screen (one at a time)
  - Broadcast STOP_SCREEN_SHARE message when another participant wants to take over
- **REQ-3.7:** The Chat Panel shall:
  - Display scrollable chat messages with sender name, timestamp, and content
  - Send messages via LiveKit DataChannel for real-time delivery
  - Provide chat input field with Send button
  - Auto-scroll to latest messages
- **REQ-3.8:** The system shall implement a Floating Dock component at the bottom center:
  - Rounded pill shape with backdrop blur and shadow
  - Buttons for: Mic toggle, Camera toggle, Screen Share toggle, Whiteboard toggle, Chat toggle, Participants toggle, Leave Room
  - "End Meeting" button (red) visible only to Room Host
  - Visual feedback: violet background for active states, red for destructive actions
  - Confirmation dialogs for Leave Room and End Meeting actions
- **REQ-3.9:** The system shall handle meeting end scenarios:

- Host clicks "End Meeting": Broadcast ROOM_ENDED message, show overlay to all participants
- Meeting Ended Overlay: Display "Meeting has ended" message with host name, countdown timer, auto-redirect to dashboard
- Participant clicks "Leave Room": Disconnect from LiveKit, redirect to dashboard

Module 4: Collaborative Whiteboard

- **REQ-4.1:** The system shall provide a collaborative whiteboard using tldraw library:
  - Display tldraw canvas when whiteboard is toggled in the Stage area
  - Provide drawing tools: pen, highlighter, eraser, shapes, text, and selection
  - Support color and stroke width customization
  - Enable image upload with server-side storage (avoids 64KB WebRTC limit)
- **REQ-4.2:** The system shall synchronize whiteboard state in real-time:
  - Broadcast drawing operations to all participants via LiveKit DataChannel within 100ms
  - Apply remote updates to local canvas immediately upon receipt
  - Use timestamp-based last-write-wins conflict resolution for concurrent edits
  - Synchronize current whiteboard state to new participants joining the room
- **REQ-4.3:** The system shall persist whiteboard state:
  - Save whiteboard snapshot to localStorage for panel toggle persistence during session
  - Load whiteboard state from database when joining a room (fallback when no other participants)
  - Allow host/co-host to save whiteboard snapshot to database via API
- **REQ-4.4:** The system shall support whiteboard permission control:
  - Host can set whiteboard permission to: open (all can edit), restricted (only allowed users), or disabled (hidden)
  - Host can grant/revoke whiteboard access to specific participants
  - Display whiteboard in read-only mode for users without edit permission
  - Hide whiteboard completely when permission is set to disabled

Module 5: Personal Notes

- **REQ-5.1:** The system shall provide a personal notes editor using Tiptap library:
  - Display rich-text editor in the Notes tab of the Chat/Notes panel
  - Each user has their own private notes (not shared with other participants)
  - Support rich text formatting: bold, italic, strikethrough
  - Support heading levels (H1, H2, H3) and paragraph formatting
  - Support bullet lists and numbered lists
- **REQ-5.2:** The system shall persist personal notes:
  - Auto-save notes content to database via /api/room/[roomId]/notes endpoint
  - Load saved notes when user rejoins the same room
  - Each user's notes are stored separately per room (RoomNote model)
- **REQ-5.3:** The system shall support notes permission control:
  - Host can set notes permission to: open (all can use), restricted (only allowed users), or disabled (hidden)
  - Host can grant/revoke notes access to specific participants
  - Display notes in read-only mode for users without edit permission
  - Hide notes tab completely when permission is set to disabled

Module 6: Session History

- **REQ-6.1:** The system shall provide a Sessions page (/sessions) displaying:
  - Grid of past session cards with room name, date, duration, and participant count
  - Filter and browse through session history
  - Navigate to session detail page by clicking on session cards
- **REQ-6.2:** The system shall provide a Session Detail page (/sessions/[id]) with:
  - Sticky header showing session title, date, time, and duration
  - Two-column layout: left (2/3 width) with tabs for Whiteboard and Notes; right (1/3 width) with Participants and Session Statistics cards
- **REQ-6.3:** The Whiteboard tab shall display:
  - Saved whiteboard snapshot as a static image (PNG)
  - Interactive viewer with zoom and pan controls
  - Placeholder message if no whiteboard snapshot was saved

2.2 Use case Diagram

2.2.1 System-level Use Case Diagram

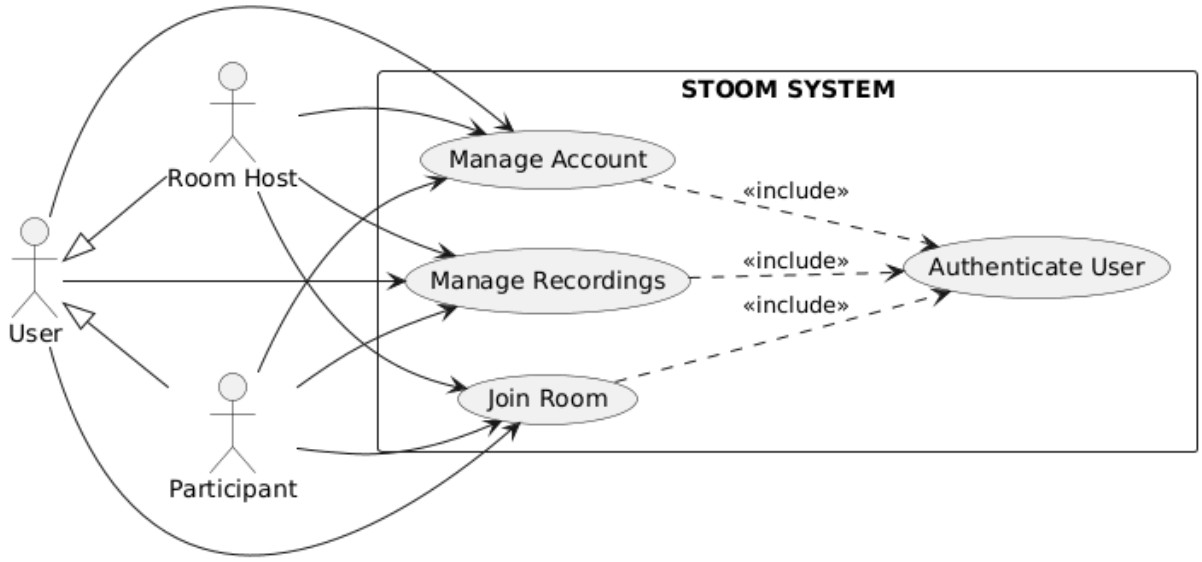

Figure 2-1. System-level Use case Diagram

2.2.2 Detailed Use Case: Manage Account

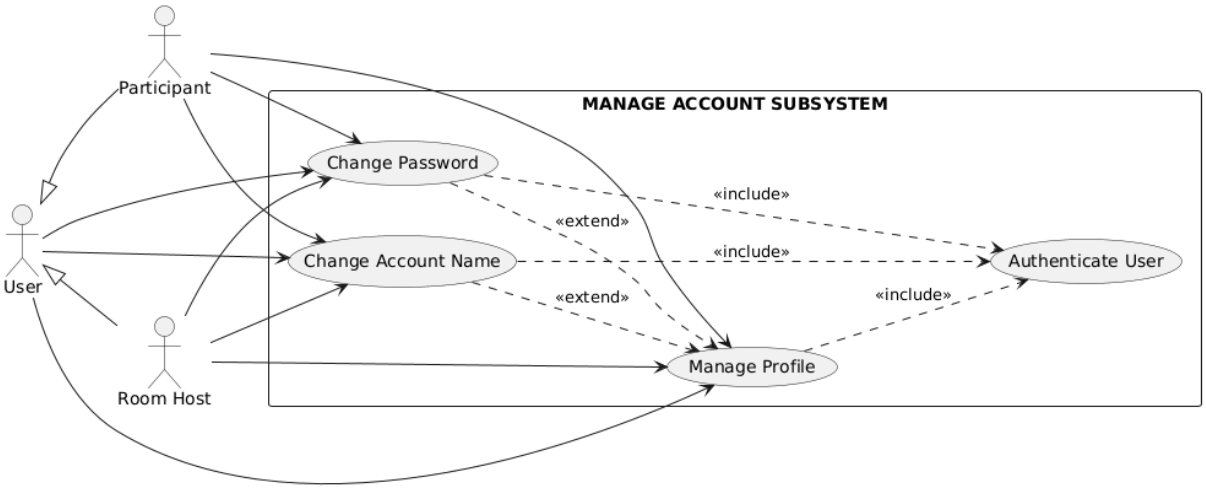

Figure 2-2. Detailed Use Case: Manage Account

2.2.3 Detailed Use Case: Join Room

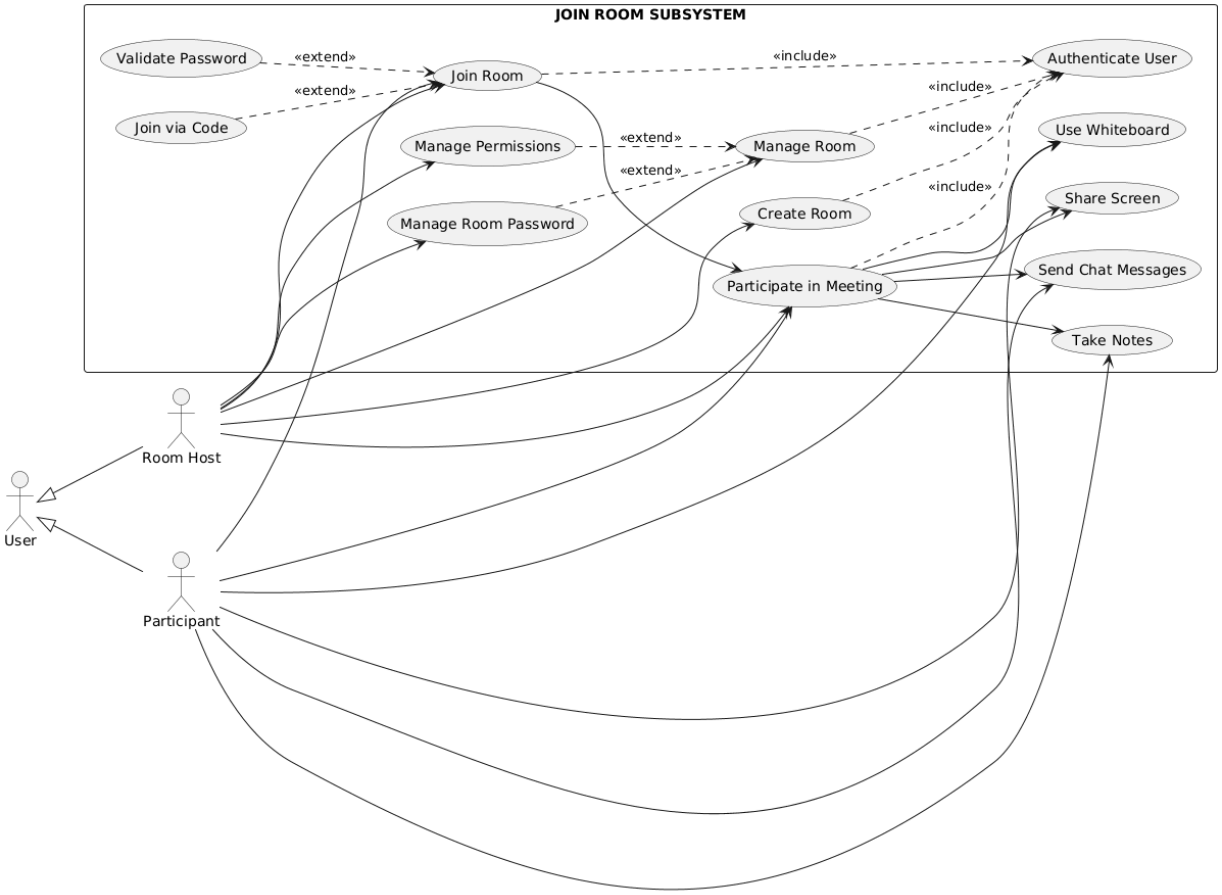

Figure 2-3. Detailed Use Case: Join Room

2.2.4 Detailed Use Case: View Sessions

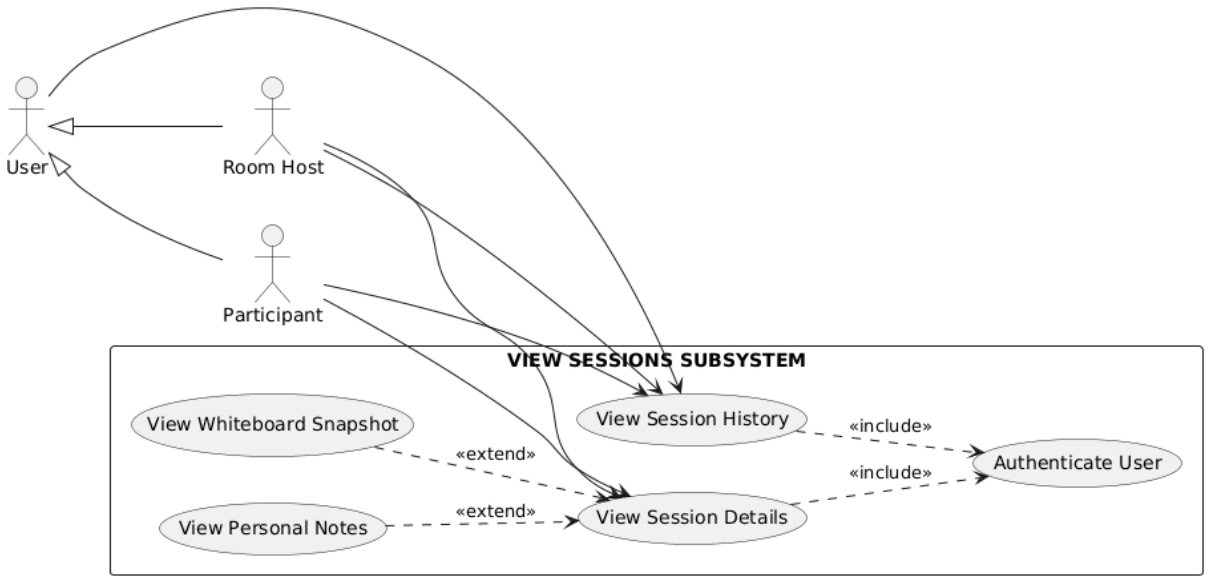

Figure 2-4. Detailed Use Case: View Sessions

2.2.5 Detailed Use Case Specifications

Use Case 0: Authenticate User

Attribute	Description
Use Case Name	Authenticate User
Actor	User (base actor)
Description	A user authenticates with the system through Clerk authentication service, supporting email/password and social login methods (Google, GitHub). Upon successful authentication, the user gains access to protected features of the platform.
Preconditions	1. User has a valid account or wishes to create one 2. User is on the sign-in or sign-up page 3. Clerk authentication service is accessible
Postconditions	1. User is authenticated and has an active session 2. User is redirected to /dashboard 3. User can access protected routes and features 4. User's authentication status is maintained across the session
Flow	**Main Flow:** 1. User navigates to /sign-in or /sign-up page 2. System displays Clerk authentication interface with custom violet-themed styling 3. User selects authentication method (email/password or social login) 4. If email/password: User enters email and password, then clicks "Sign In" 5. If social login: User clicks social provider button (Google/GitHub) and authorizes 6. Clerk validates user credentials or processes social login 7. If credentials are valid, Clerk creates/updates user session 8. System receives authentication success response from Clerk 9. System redirects user to /dashboard 10. User's authentication status is stored in session **Alternative Flow 6a:** If credentials are invalid: 6a.1. Clerk returns authentication error 6a.2. System displays error message "Invalid credentials" 6a.3. User can retry authentication or reset password **Alternative Flow 4a:** If user selects "Sign Up" instead: 4a.1. User is redirected to /sign-up page 4a.2. User enters email, password, and confirms password 4a.3. Clerk creates new user account 4a.4. Continue with step 7 **Alternative Flow 7a:** If social login authorization is denied:

	7a.1. User is returned to sign-in page 7a.2. User can try again or use email/password method

Use Case 1: Join Room

Attribute	**Description**
Use Case Name	Join Room
Actor	Participant (specialized User)
Description	A user joins an existing meeting room by entering a room code or navigating directly to /room/[roomCode]. The system validates room existence and status via /api/room/validate, displays the pre-join screen with camera/mic preview, validates room password if required, requests a LiveKit token, and connects to the LiveKit room for real-time video/audio communication.
Preconditions	1. User is authenticated via Clerk 2. Room with the provided code exists in the database 3. Room status is WAITING or ACTIVE (not ENDED) 4. Room isActive flag is true 5. LiveKit server is running and accessible
Postconditions	1. User is connected to LiveKit room with video/audio streams 2. User's data is synced to database via token API (upsert) 3. RoomParticipant record is created via LiveKit webhook 4. Room status transitions to ACTIVE (via room_started webhook) 5. User can see other participants and interact with room features
Flow	**Main Flow:** 1. User navigates to /room/[roomCode] (via dashboard or direct link) 2. System calls /api/room/validate?code=[roomCode] to check room status 3. API validates: room exists, status is not ENDED, isActive is true 4. API performs stale room cleanup (checks LiveKit room existence) 5. API returns room info including hasPassword flag 6. System displays pre-join screen with video preview and media controls 7. If room has password and user is not host, system shows password input field 8. User optionally toggles microphone and camera 9. User enters password (if required) and clicks "Join Room" button 10. System calls /api/livekit/token with roomName and password (if provided) 11. Token API validates password using bcrypt comparison (host

	is exempt) 12. Token API upserts user data to database (clerkId, name, email, imageUrl) 13. Token API generates LiveKit access token with user identity and metadata 14. Client connects to LiveKit room using token 15. LiveKit sends room_started webhook (if first participant) → Room status = ACTIVE 16. LiveKit sends participant_joined webhook → RoomParticipant record created 17. System loads room data: permissions via /api/room/[roomId]/permissions, chat history 18. System initializes collaboration features based on permissions (whiteboard, notes) 19. System displays main room interface with participants sidebar, video feeds, and controls **Alternative Flow 3a:** If room does not exist: 3a.1. API returns 404 with code "ROOM_NOT_FOUND" 3a.2. System displays "Room Not Found" error page 3a.3. User can navigate back to dashboard **Alternative Flow 3b:** If room has ended: 3b.1. API returns 410 with code "ROOM_ENDED" 3b.2. System displays "Meeting Ended" error page 3b.3. User can navigate back to dashboard **Alternative Flow 4a:** If stale room detected (ACTIVE but no LiveKit room for 5+ minutes): 4a.1. API marks room as ENDED and returns 410 4a.2. Continue with Alternative Flow 3b **Alternative Flow 11a:** If password is required but not provided: 11a.1. Token API returns 401 with code "PASSWORD_REQUIRED" 11a.2. System displays password input field 11a.3. User enters password and retries **Alternative Flow 11b:** If password is incorrect: 11b.1. Token API returns 401 with code "INVALID_PASSWORD" 11b.2. System displays "Incorrect password" error message 11b.3. User can retry with correct password or cancel

Use Case 2: Collaborate on Whiteboard

Attribute	Description

Use Case Name	Collaborate on Whiteboard
Actor	Participant (specialized User)
Description	A user draws, writes, or adds shapes on the collaborative whiteboard. The system updates the local canvas optimistically, synchronizes changes with other participants in real-time through LiveKit DataChannel, and persists the final state when saved by the host.
Preconditions	1. User is in an active room (/room/[roomId]) 2. Whiteboard is visible in Main Stage 3. User has permission to edit whiteboard (based on room permission settings) 4. LiveKit DataChannel connection is established
Postconditions	1. User's drawing appears on local canvas immediately 2. Other participants see the drawing within 100ms network latency 3. Whiteboard state is synchronized across all clients 4. Whiteboard snapshot can be saved to database by host
Flow	**Main Flow:** 1. User clicks whiteboard toggle button in Floating Dock or Stage control bar 2. System displays whiteboard in Main Stage (if not already visible) 3. User selects drawing tool (pen, eraser, shape, text) from tldraw toolbar 4. User performs drawing action (mouse down, drag, mouse up) 5. Client application (tldraw) updates local canvas optimistically 6. Client creates operation message containing: record changes, timestamp, senderId 7. Client sends operation message via LiveKit DataChannel 8. Other clients receive operation and apply changes using timestamp-based conflict resolution 9. All clients display updated whiteboard state 10. Steps 4-9 repeat for each drawing action **Alternative Flow 3a:** If whiteboard is not visible: 3a.1. System shows whiteboard in Main Stage 3a.2. Continue with step 3 **Alternative Flow 7a:** If DataChannel connection is lost: 7a.1. Client shows "Disconnected" indicator 7a.2. Client attempts to reconnect 7a.3. Upon reconnection, client requests sync from other participants 7a.4. Continue with step 8

	Alternative Flow 8a: If user does not have edit permission: 8a.1. Whiteboard displays in read-only mode 8a.2. User can view but not modify the whiteboard 8a.3. "View only" indicator is displayed

Use Case 3: Take Personal Notes

Attribute	**Description**
Use Case Name	Take Personal Notes
Actor	Participant (specialized User)
Description	A user takes personal notes during a meeting using the Tiptap rich-text editor. Notes are private to each user and automatically saved to the database. Users can format text with headings, lists, and text styles.
Preconditions	1. User is in an active room (/room/[roomId]) 2. Notes tab is accessible in the Chat/Notes panel 3. User has permission to use notes (based on room permission settings) 4. Database connection is available
Postconditions	1. User's notes are displayed in the editor 2. Notes are automatically saved to database 3. Notes persist across sessions (user can view them when rejoining the room) 4. Notes are accessible in session history after meeting ends
Flow	**Main Flow:** 1. User clicks Notes tab in the Chat/Notes panel 2. System displays Tiptap rich-text editor with formatting toolbar 3. System loads any previously saved notes for this user in this room 4. User types content in the editor 5. User optionally applies formatting (bold, italic, headings, lists) 6. System auto-saves notes to database via /api/room/[roomId]/notes endpoint 7. Steps 4-6 repeat as user continues taking notes **Alternative Flow 3a:** If no previous notes exist: 3a.1. System displays empty editor with placeholder 3a.2. Continue with step 4 **Alternative Flow 6a:** If save fails: 6a.1. System retries save operation 6a.2. If persistent failure, notes remain in local state 6a.3. User is notified of save failure

	Alternative Flow 2a: If user does not have notes permission: 2a.1. Notes tab is hidden or displays read-only 2a.2. User cannot edit notes

Use Case 4: View Session History

Attribute	**Description**
Use Case Name	View Session History
Actor	User (base actor)
Description	A user views their past meeting sessions, including session details, whiteboard snapshots, and personal notes. Users can browse through their session history and access detailed information about each completed meeting.
Preconditions	1. User is authenticated via Clerk 2. User has participated in at least one meeting session 3. Session has ended (status = ENDED)
Postconditions	1. User can view list of past sessions 2. User can access session details including whiteboard and notes 3. Session data is displayed in read-only format
Flow	**Main Flow:** 1. User navigates to /sessions page 2. System queries database for rooms where user is owner or participant 3. System displays grid of session cards with room name, date, duration, participant count 4. User clicks on a session card 5. System navigates to /sessions/[id] detail page 6. System loads session data including participants, chat messages, whiteboard snapshot, and user's notes 7. System displays session header with title, date, time, and duration 8. User can switch between Whiteboard and Notes tabs 9. Whiteboard tab displays saved snapshot as static image (if available) 10. Notes tab displays user's personal notes in read-only format 11. System displays participants list and session statistics **Alternative Flow 3a:** If user has no past sessions: 3a.1. System displays empty state message 3a.2. User can navigate to dashboard to create or join a room **Alternative Flow 9a:** If no whiteboard snapshot was saved: 9a.1. System displays placeholder message "No whiteboard snapshot available"

	Alternative Flow 10a: If user has no notes for this session: 10a.1. System displays placeholder message "No notes saved for this session"

2.3 Activity Diagrams

2.3.1 Authenticate User Activity Diagram

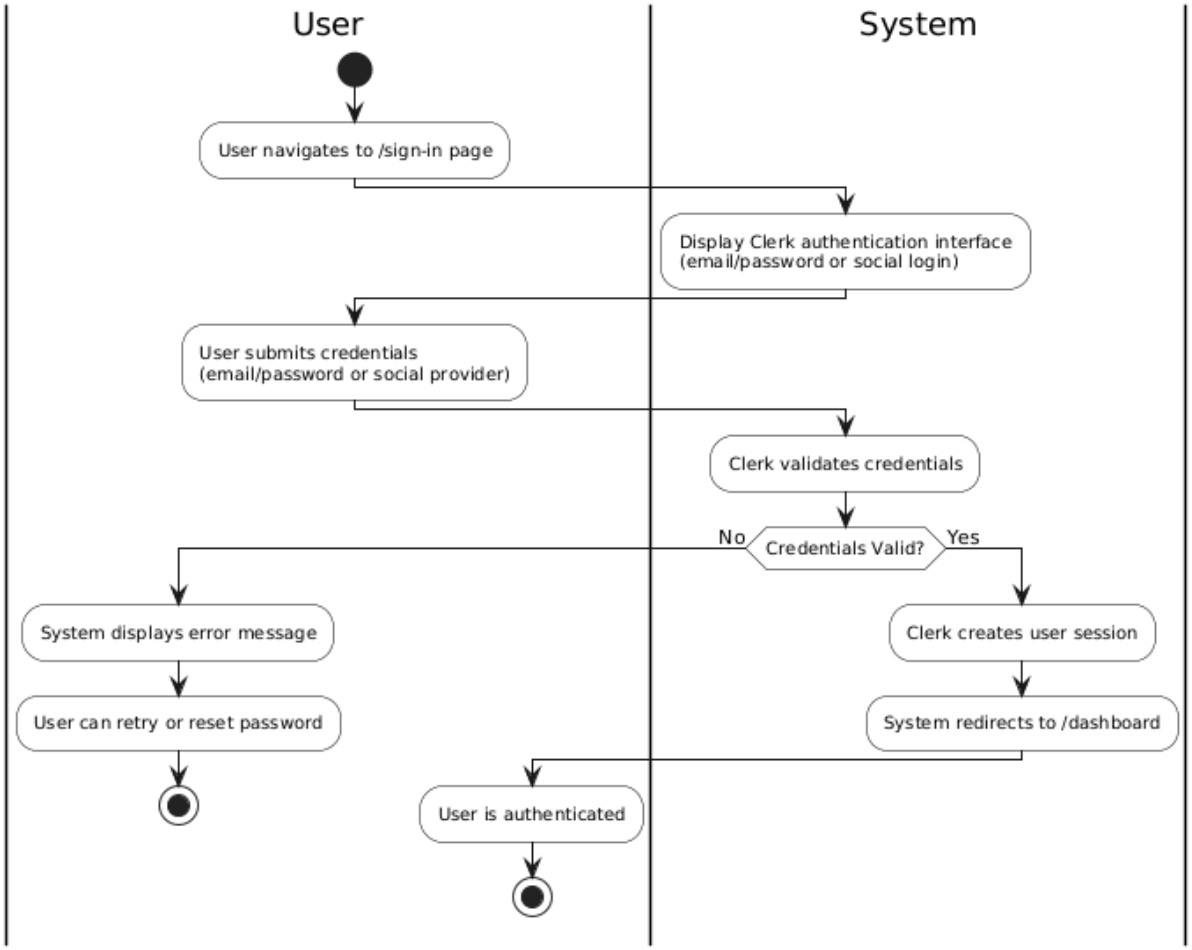

Figure 2-5. Authenticate User Activity Diagram

2.3.2 Join Room Activity Diagram

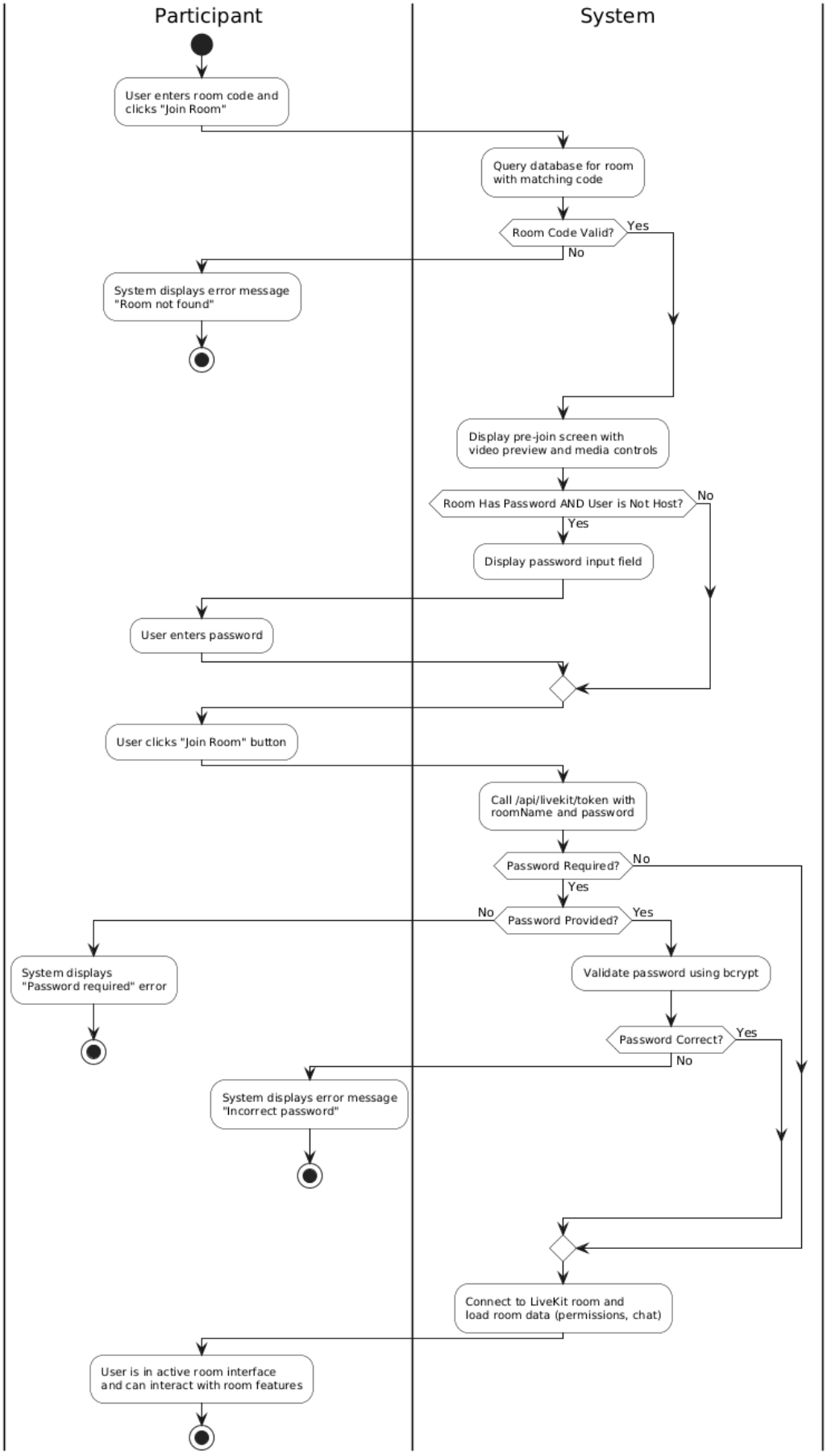

Figure 2-6. Join Room Activity Diagram

2.3.3 Collaborate on Whiteboard Activity Diagram

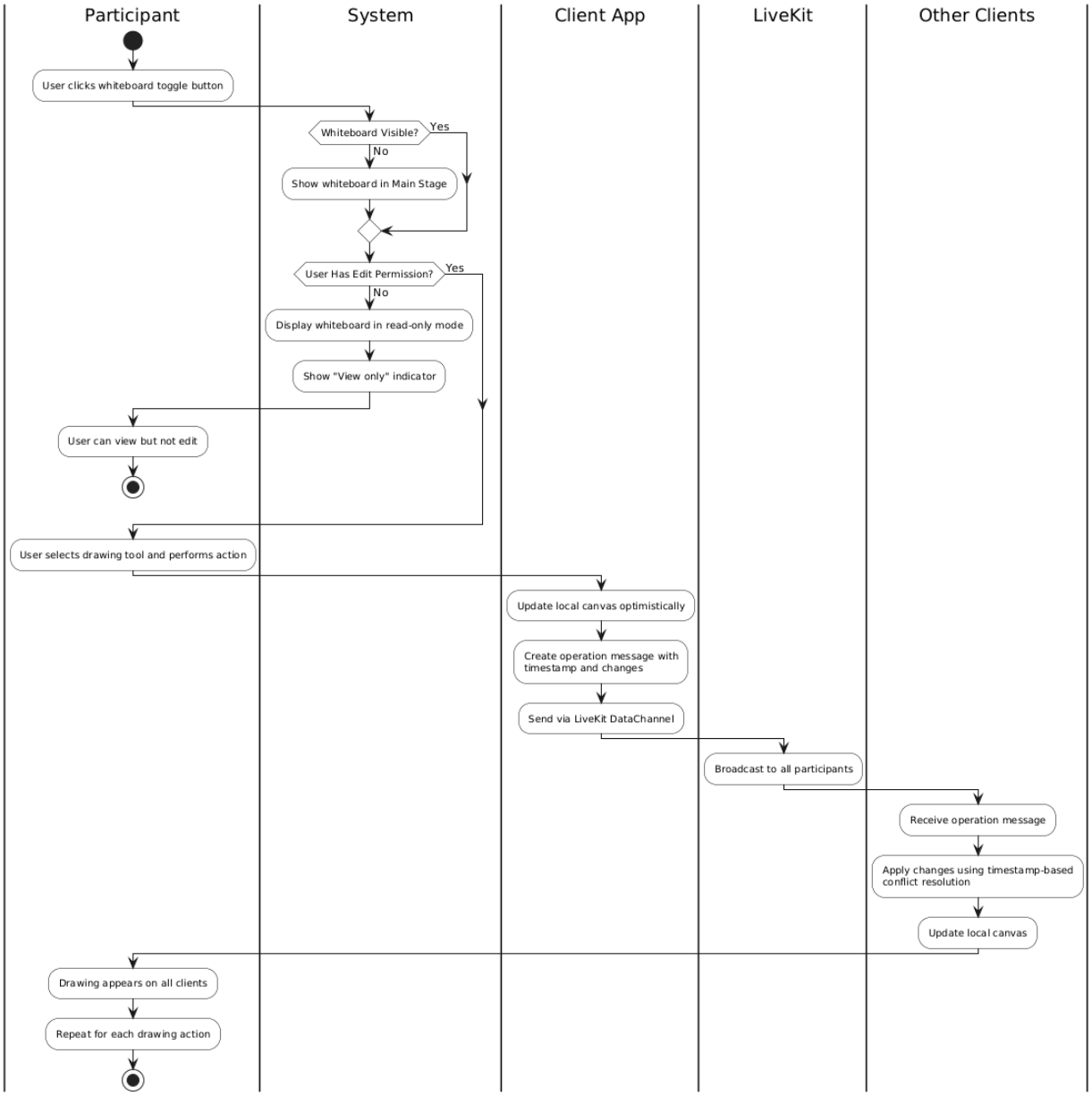

Figure 2-7. Collaborate on Whiteboard Activity Diagram

2.3.4 Take Personal Notes Activity Diagram

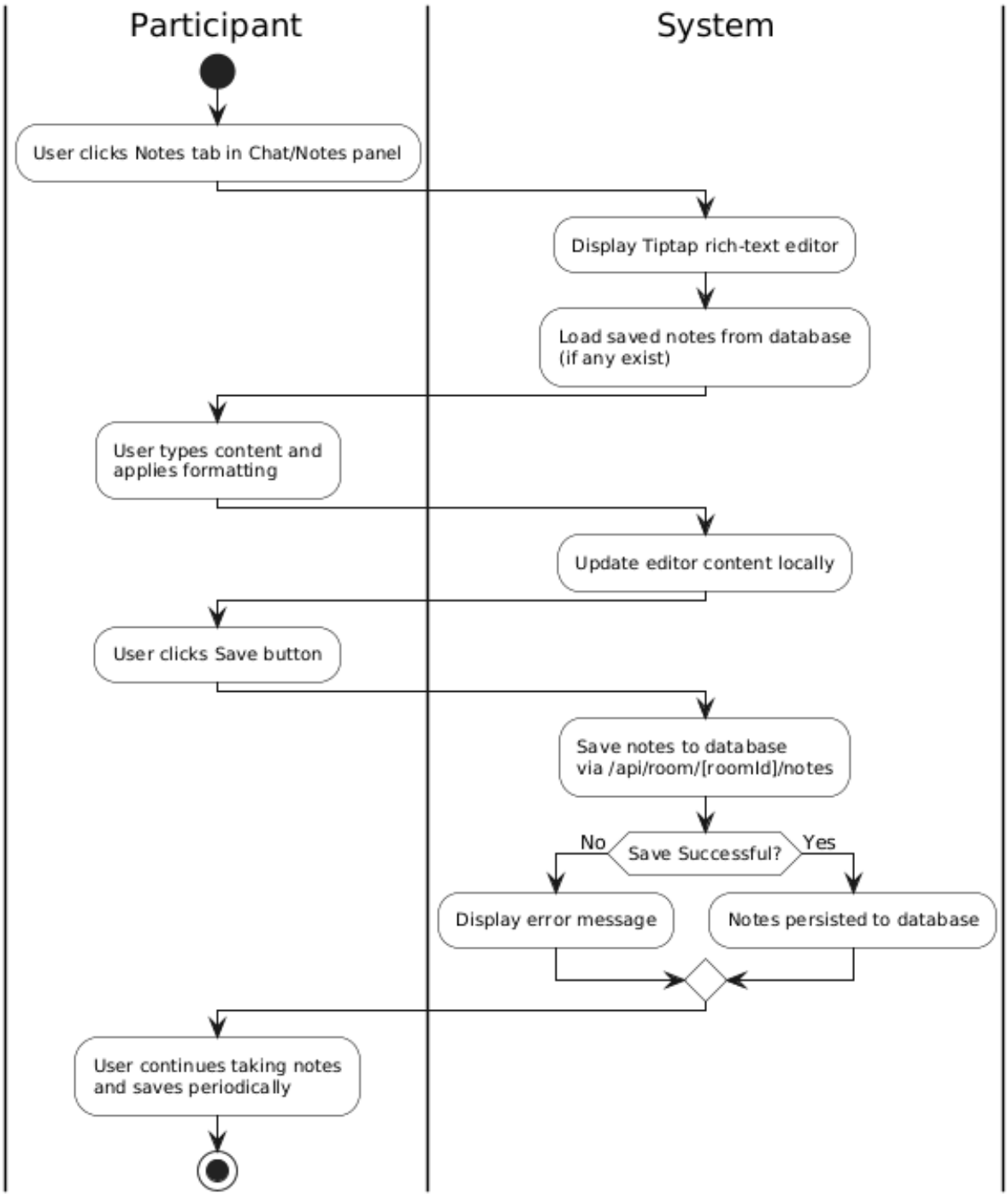

Figure 2-8. Take Personal Notes Activity Diagram

2.3.5 View Session History Activity Diagram

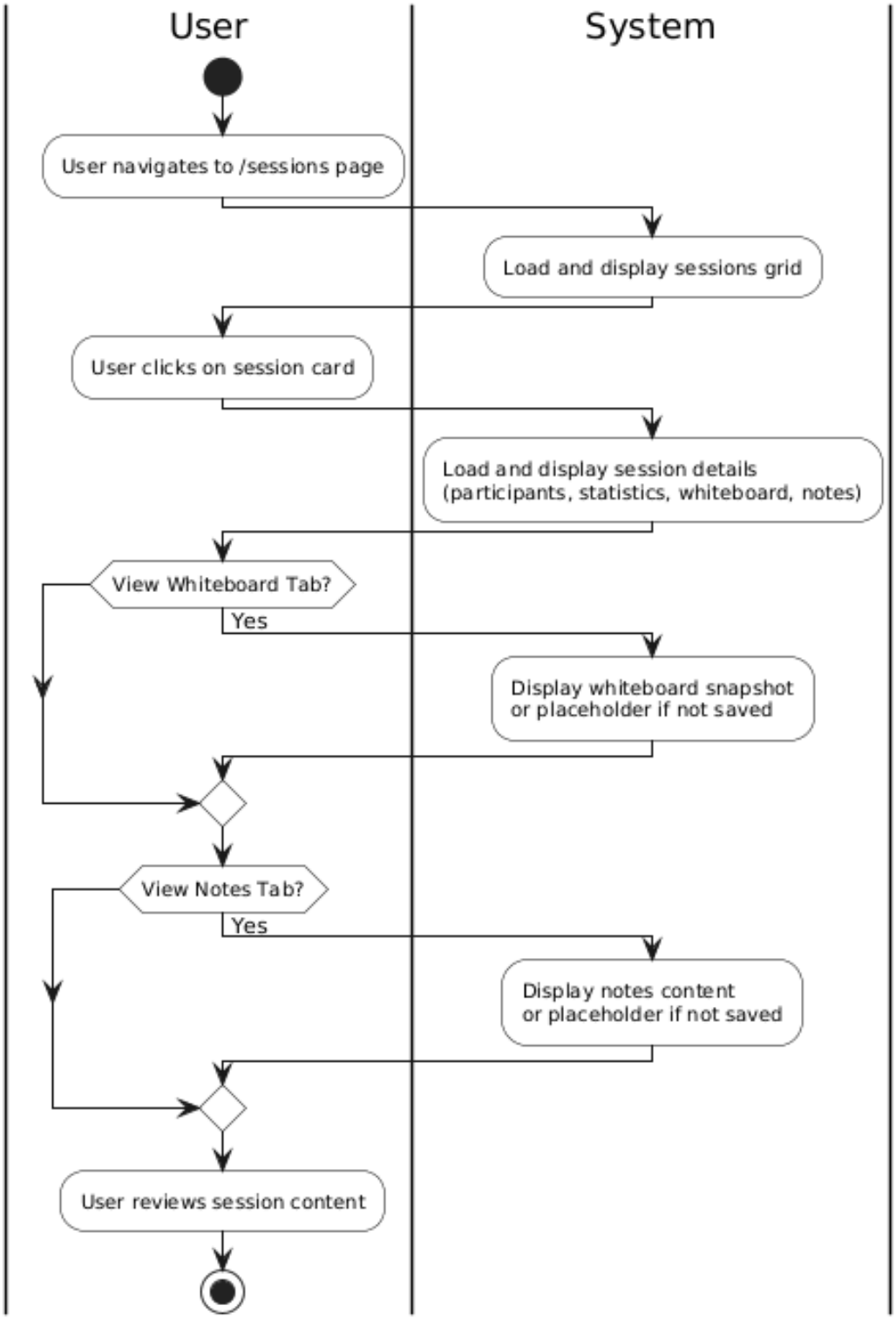

Figure 2-9. View Session History Activity Diagram

2.4 Sequence Diagrams

2.4.1 Authenticate User Sequence Diagram

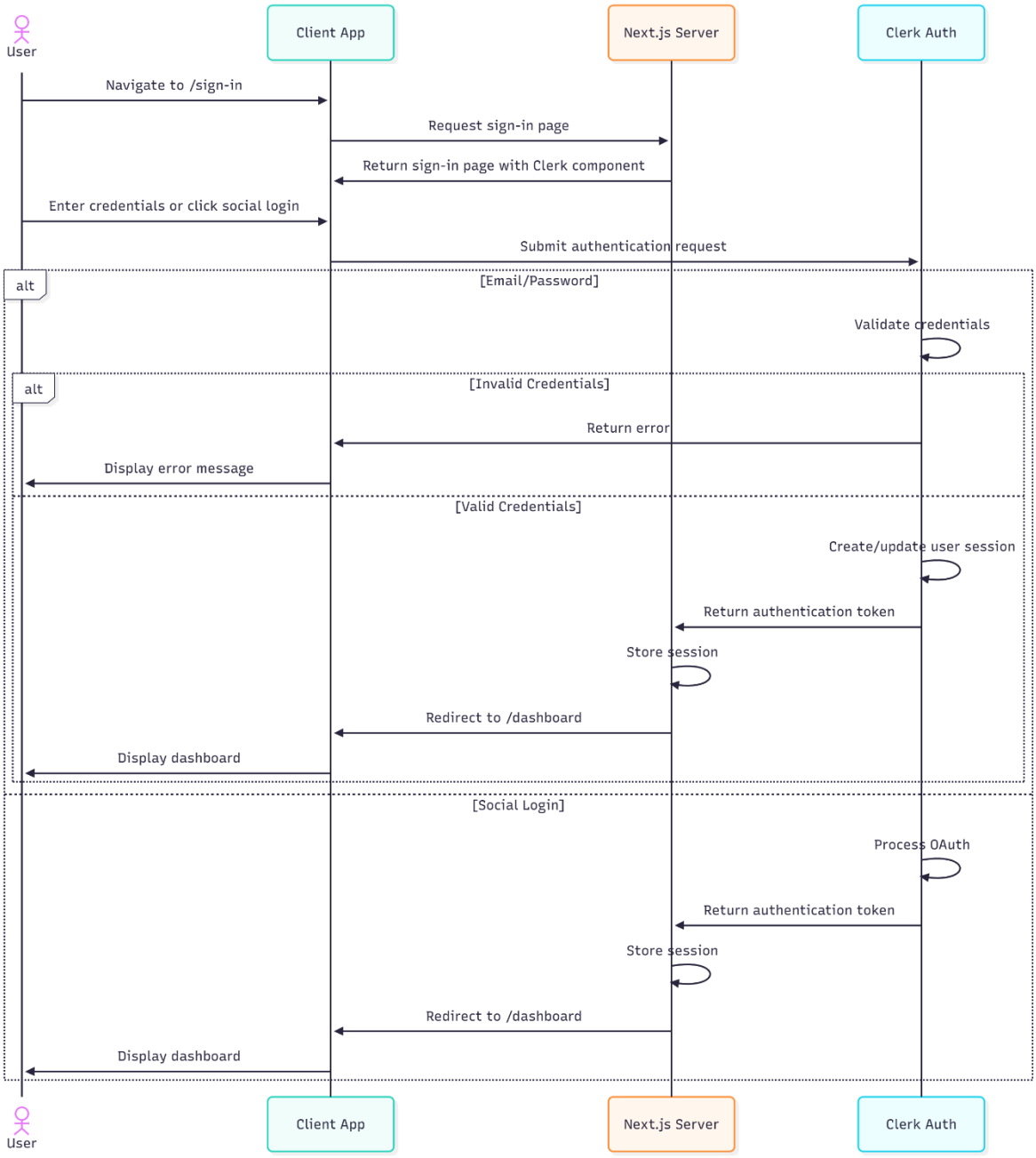

Figure 2-10. Authenticate User Sequence Diagram

2.4.2 Join Room Sequence Diagram

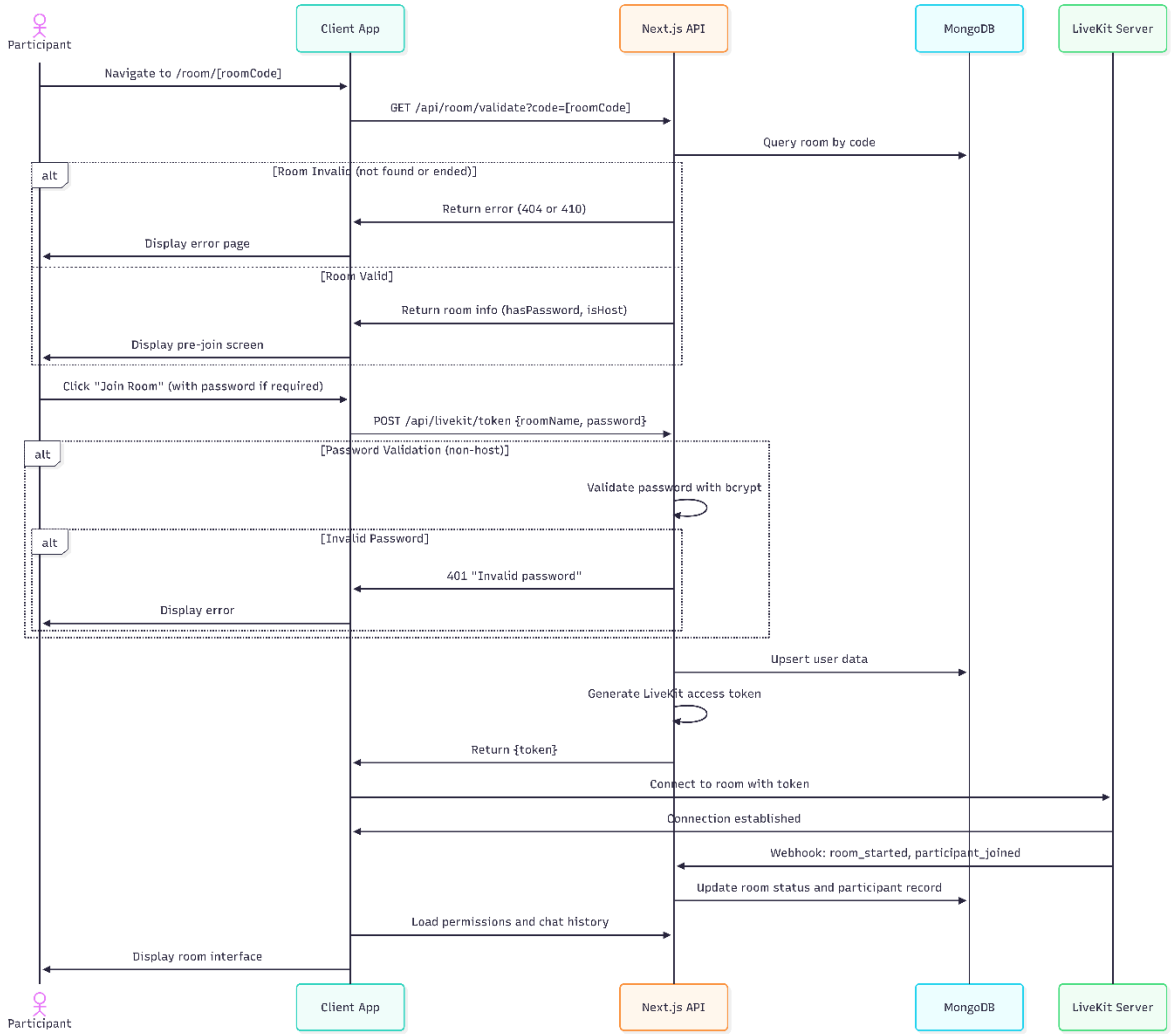

Figure 2-11. Join Room Sequence Diagram

2.4.3 Collaborate on Whiteboard Sequence Diagram

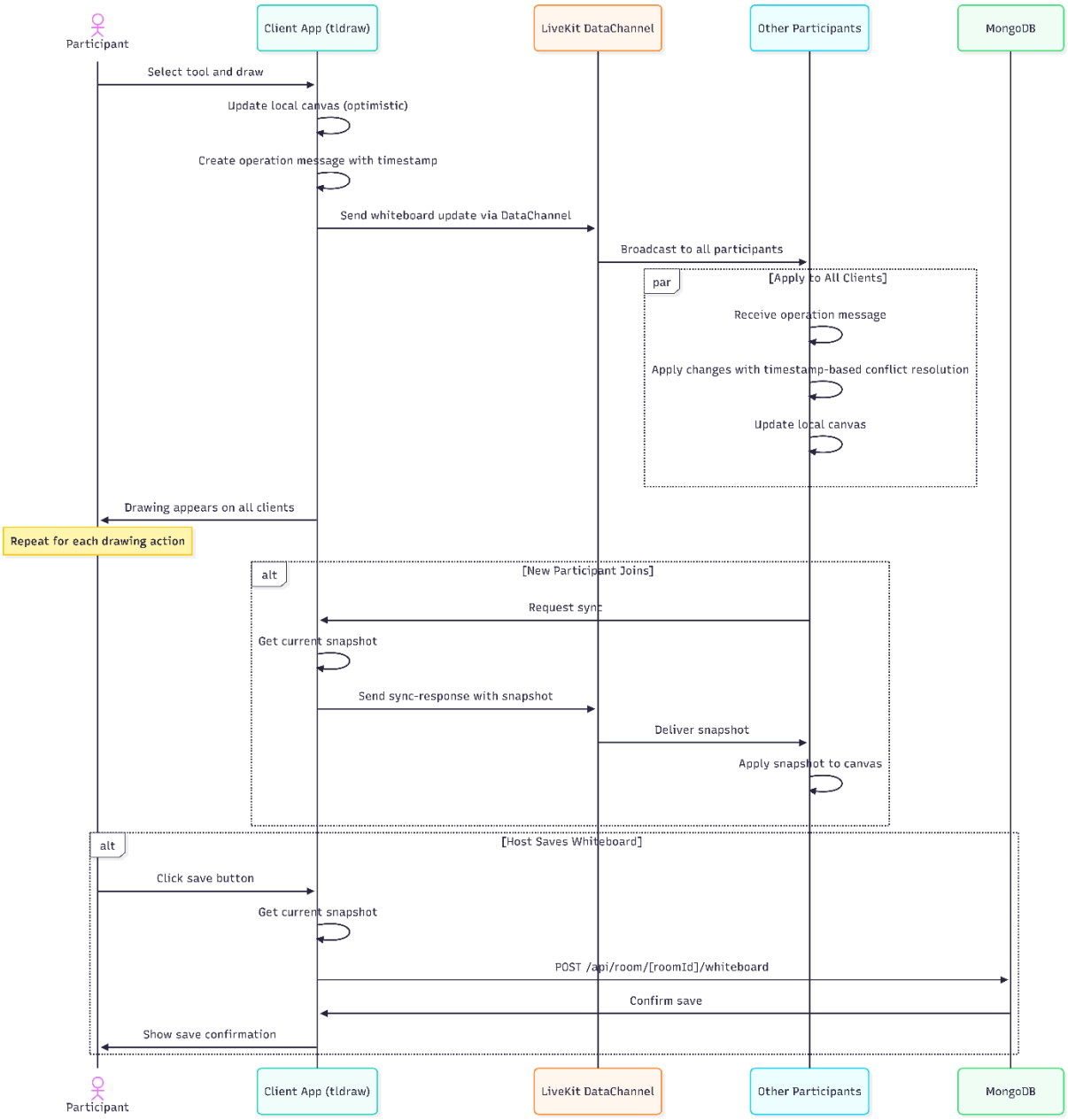

Figure 2-12. Collaborate on Whiteboard Sequence Diagram

2.4.4 Take Personal Notes Sequence Diagram

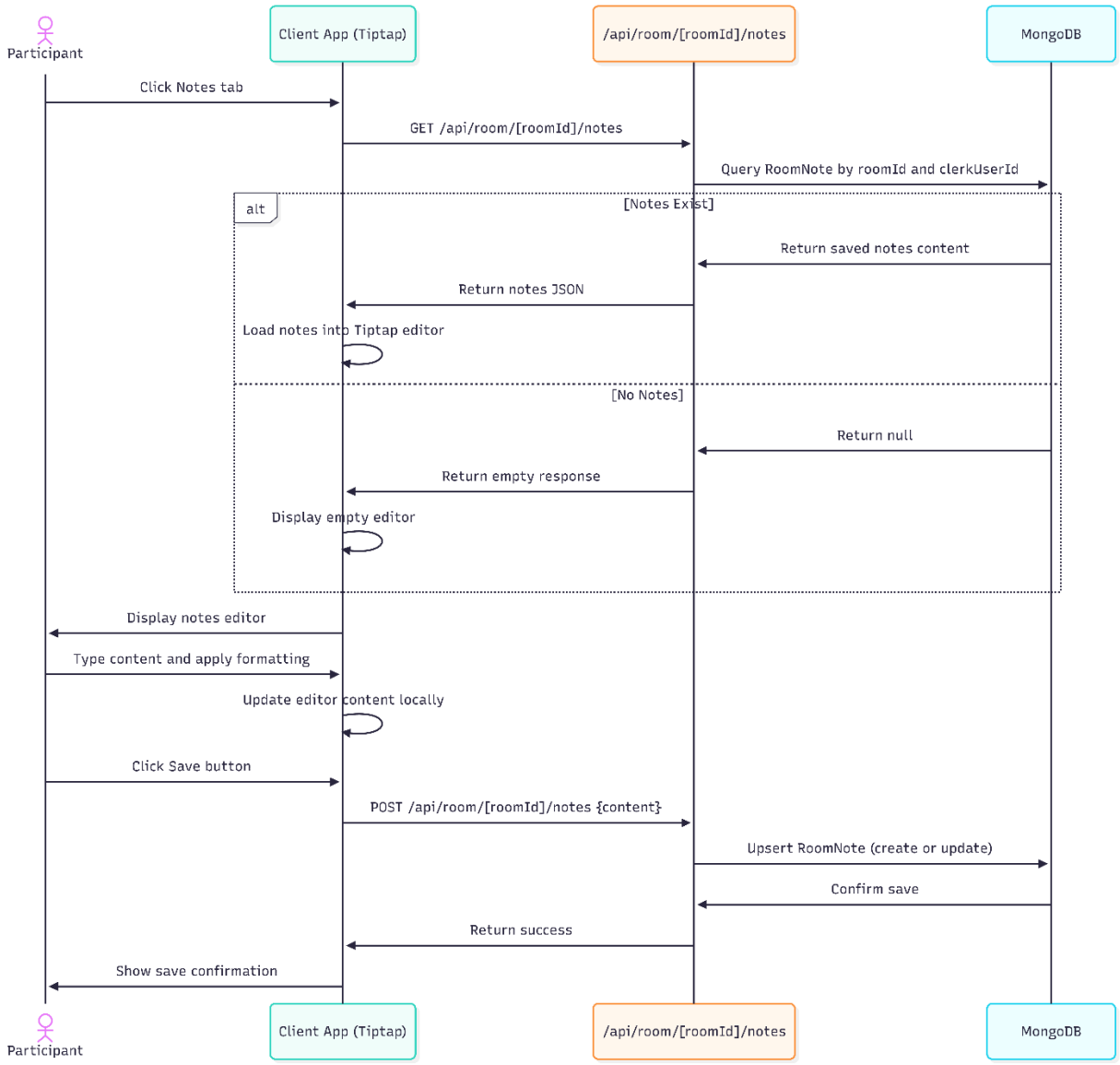

Figure 2-13. Take Personal Notes Sequence Diagram

2.4.5 View Session History Sequence Diagram

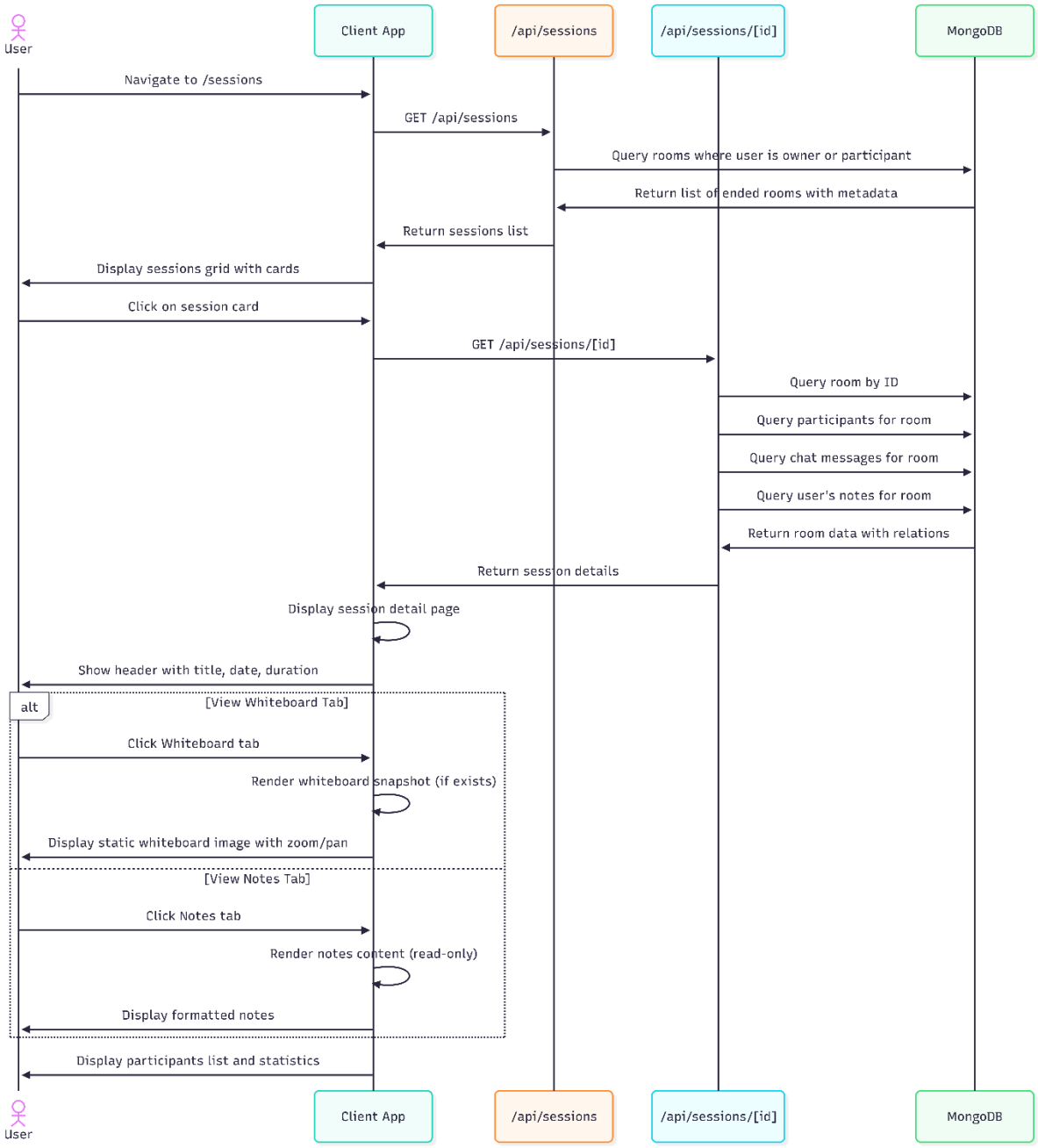

Figure 2-14. View Session History Sequence Diagram

2.5 State Diagrams

2.5.1 Room Lifecycle

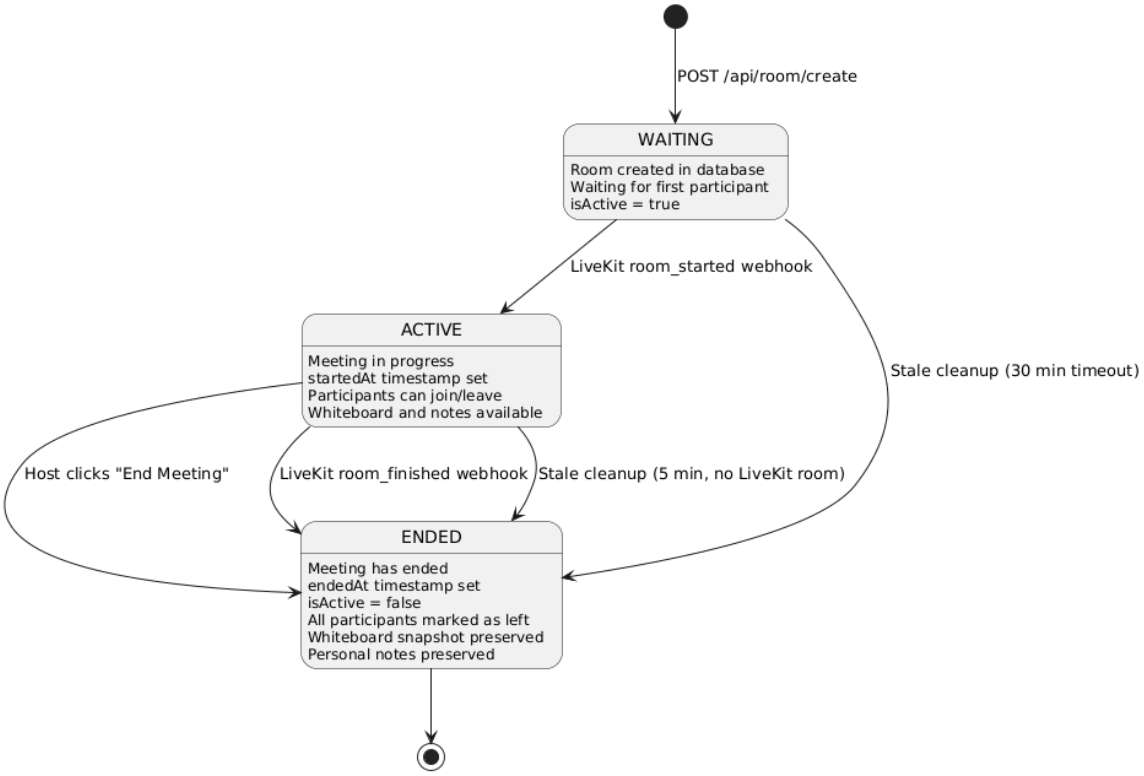

Figure 2-15. Room Lifecycle State Diagram

2.5.2 Whiteboard Collaboration State

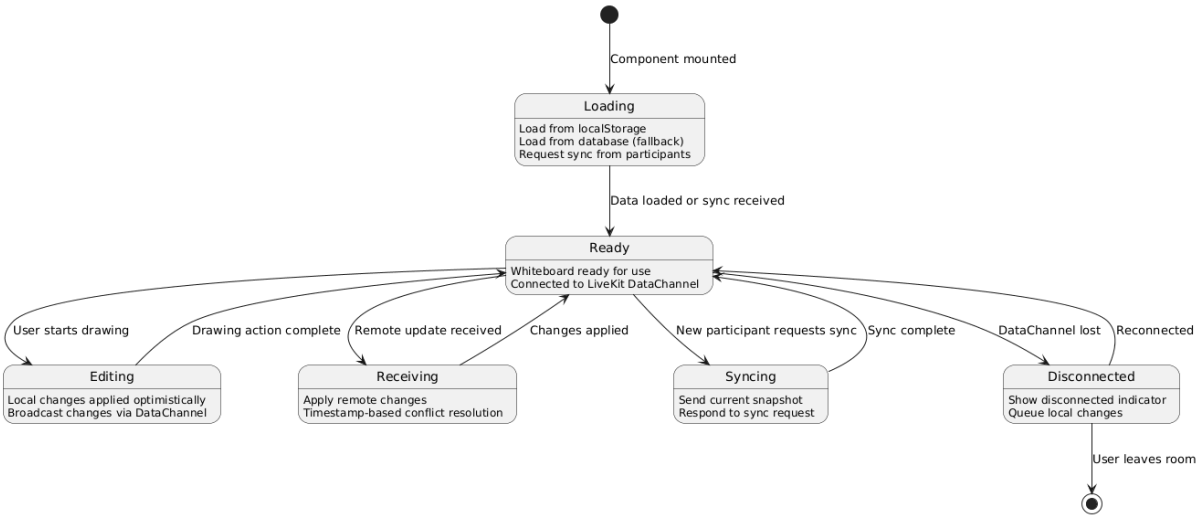

Figure 2-16. Whiteboard Collaboration State Diagram

2.5.3 Personal Notes State

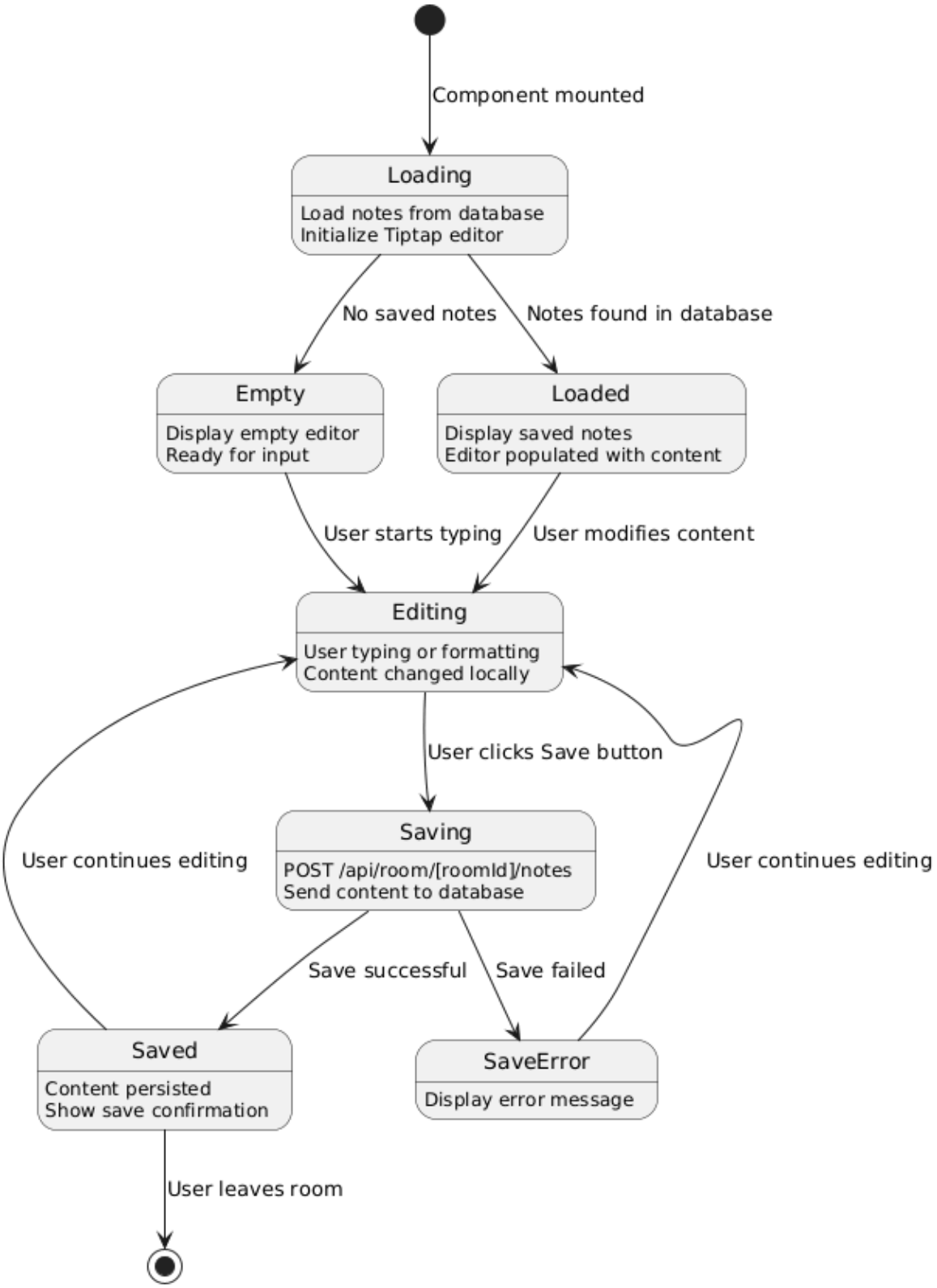

Figure 2-17. Personal Notes State Diagram

2.6 Class Diagram

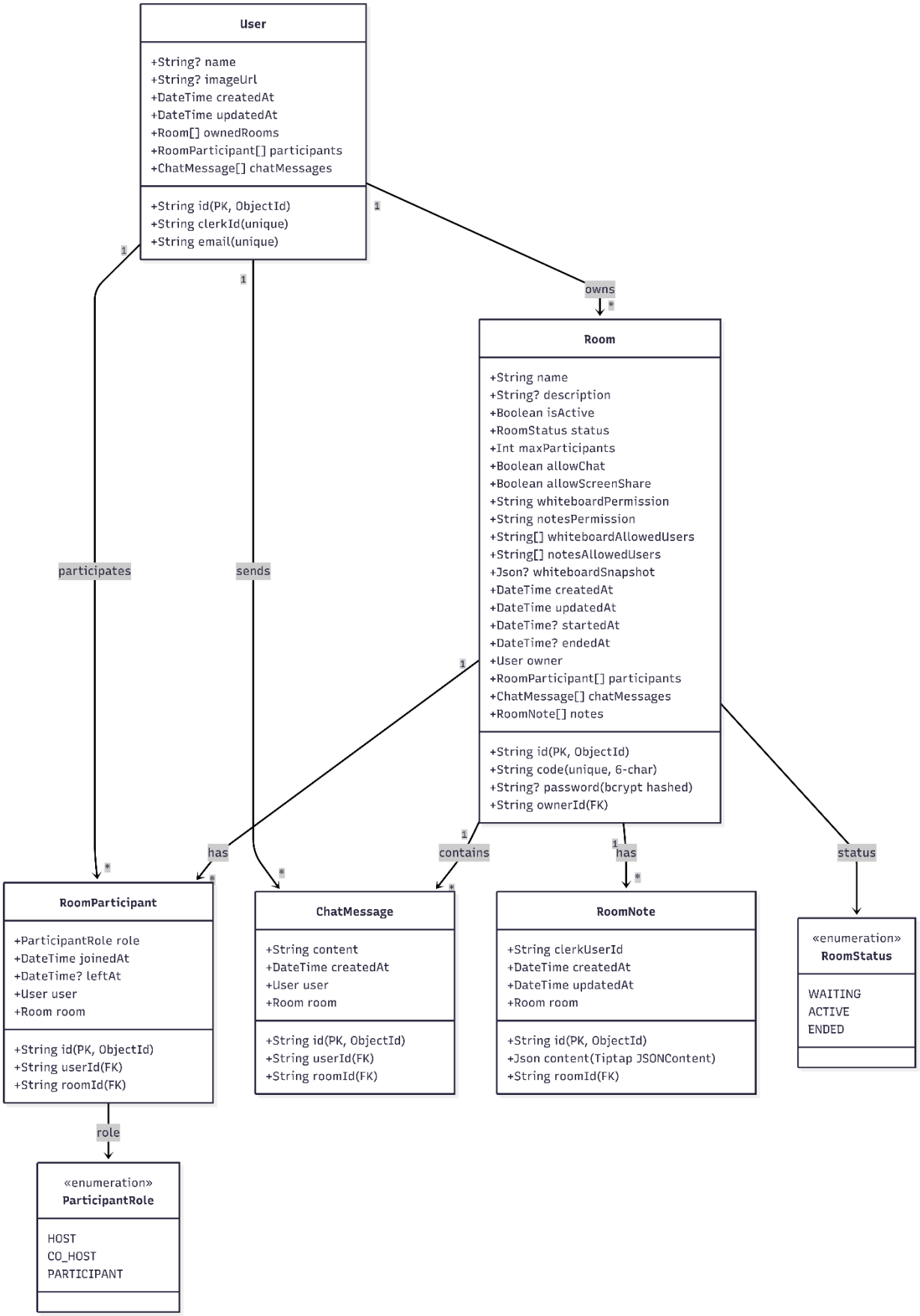

Figure 2-18. Stoom Class Diagram

Chapter 3. IMPLEMENTATION

3.1 General Interface

The The Stoom platform consists of four main interface categories, each serving distinct user needs:

- **Marketing Interface:** Public landing page introducing the platform
- **Authentication Interfaces:** Sign-in and sign-up pages for user access
- **Dashboard Interface:** Main hub for managing meetings and viewing past sessions
- **Sessions Interfaces:** Detailed views of completed sessions with whiteboard and notes
- **Room Interface:** Real-time collaboration environment with video, whiteboard, and chat

All interfaces are built using Next.js 16 App Router with route groups for logical organization. The design system uses violet as the primary brand color, with consistent styling across all pages using Tailwind CSS and shadcn/ui components..

3.2 User Interface

Landing Route:

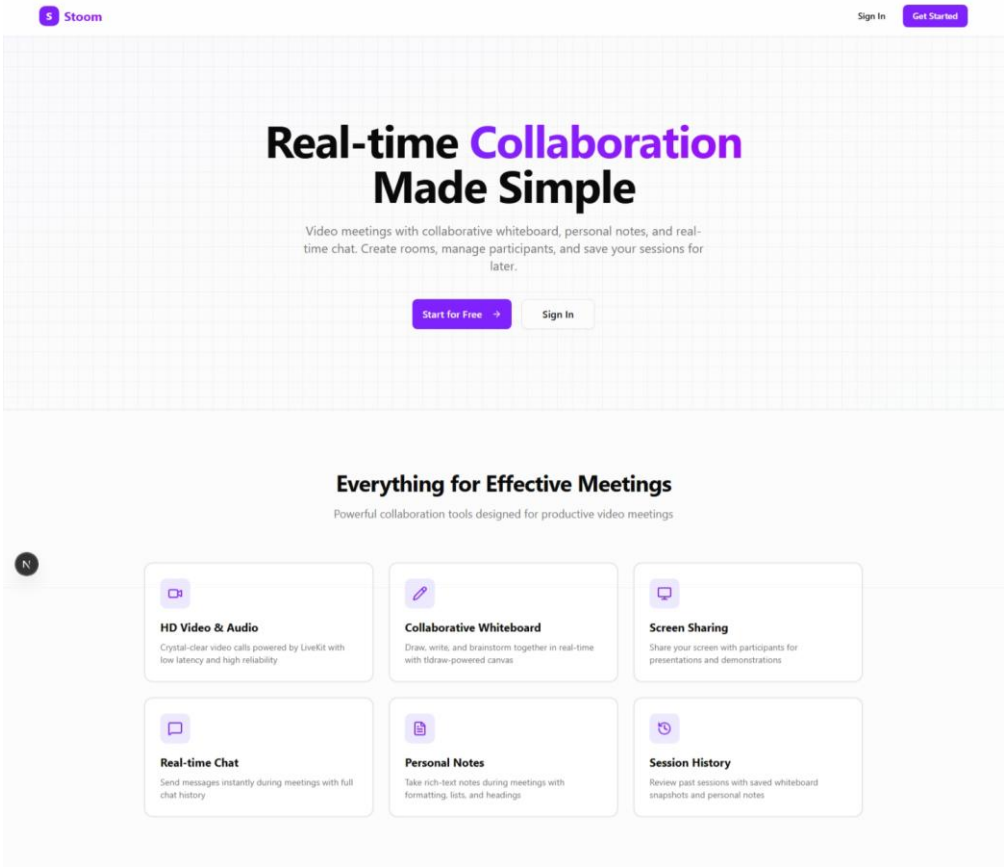

Figure 3-1. Landing Page UI

Users can browse the public landing page to learn about the platform features (Real-time Video, Collaborative Whiteboard, Personal Notes). Users can navigate to sign-in or sign-up pages to begin using the platform. No authentication required.

Authentication Routes:

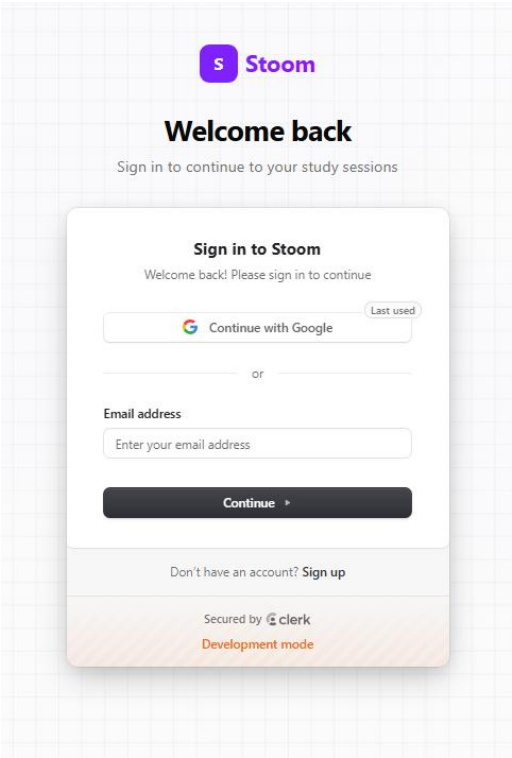

Figure 3-2. Authentication UI

Sign In (/sign-in): Users can authenticate using email/password credentials or social login providers (Google, GitHub). **Sign Up (/sign-up):** New users can create an account using email/password or social registration.

Dashboard Route

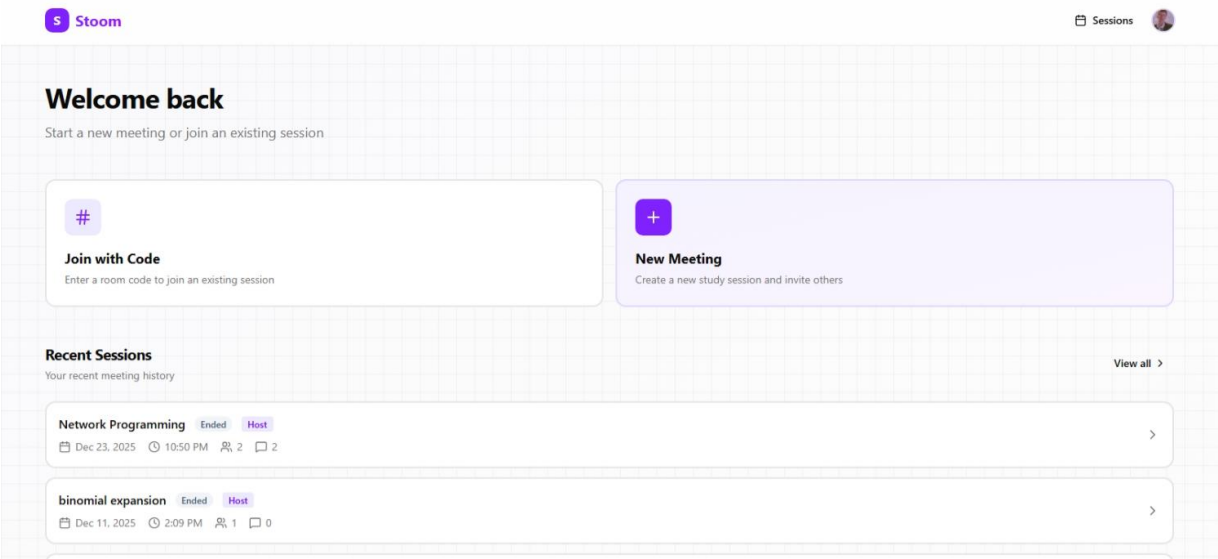

Figure 3-3. Dashboard Route

Users can switch between "Dashboard" and "Sessions" tabs. In the Dashboard tab, users can join existing rooms by entering a room code (opens dialog, navigates to room), create new meetings with custom settings (meeting title, mute on join, camera off, password protection, whiteboard restriction), and view recent sessions in a grid. Users can click "View Details" on any session card to navigate to the session detail page. In the Sessions tab, users can browse all their past session history in a grid layout and access detailed views by clicking session cards. Users can also access their profile settings via the user button in the header.

Sessions Routes:

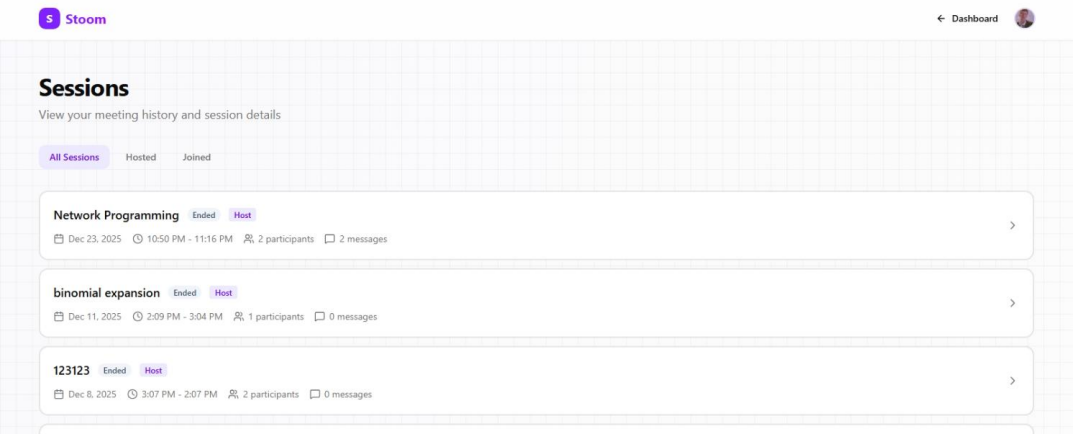

Figure 3-4. Sessions Management UI

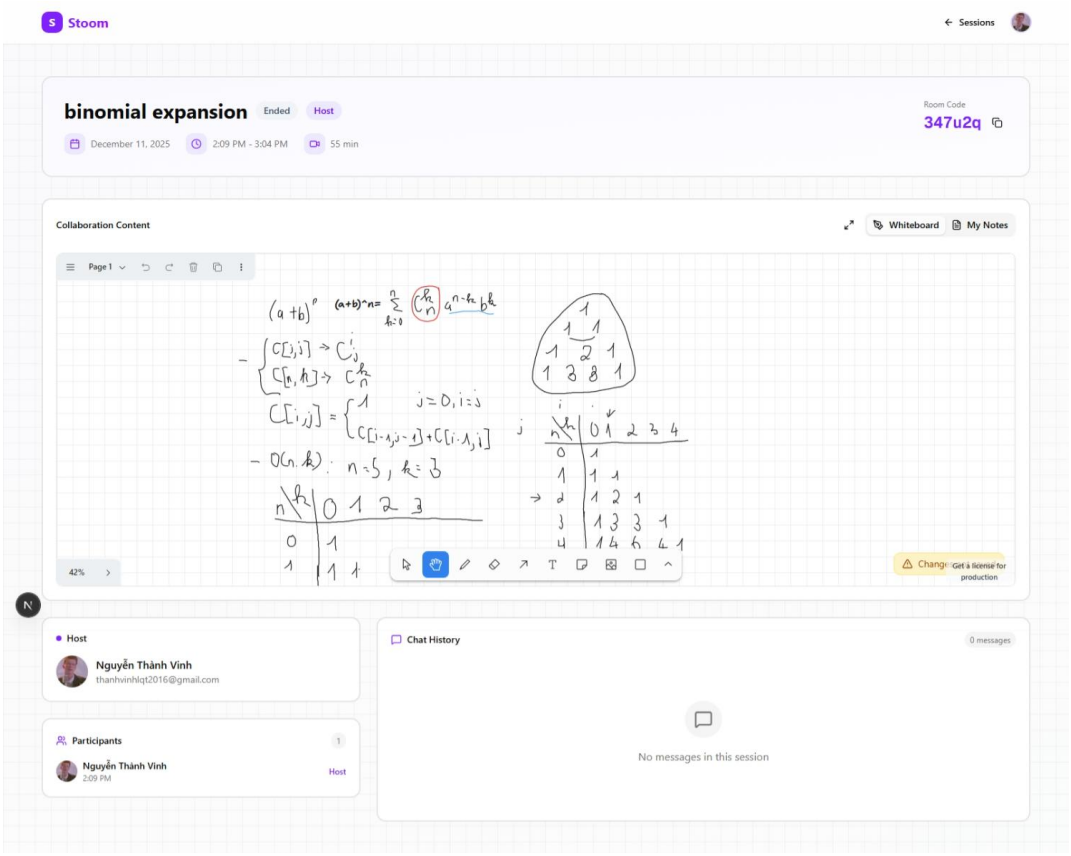

Figure 3-5. Recording Details UI

Users can view all their past session history in a grid, filter and browse through their history, and navigate to detailed views by clicking on session cards. Detail

(/sessions/[id]): Users can view comprehensive session information including whiteboard snapshots (if saved), personal notes from the session, participant list with role indicators, and session statistics (duration, participant count, message count). Users can navigate back to the sessions list. Users can switch between Whiteboard and Notes tabs to view different aspects of the session.

Room Route

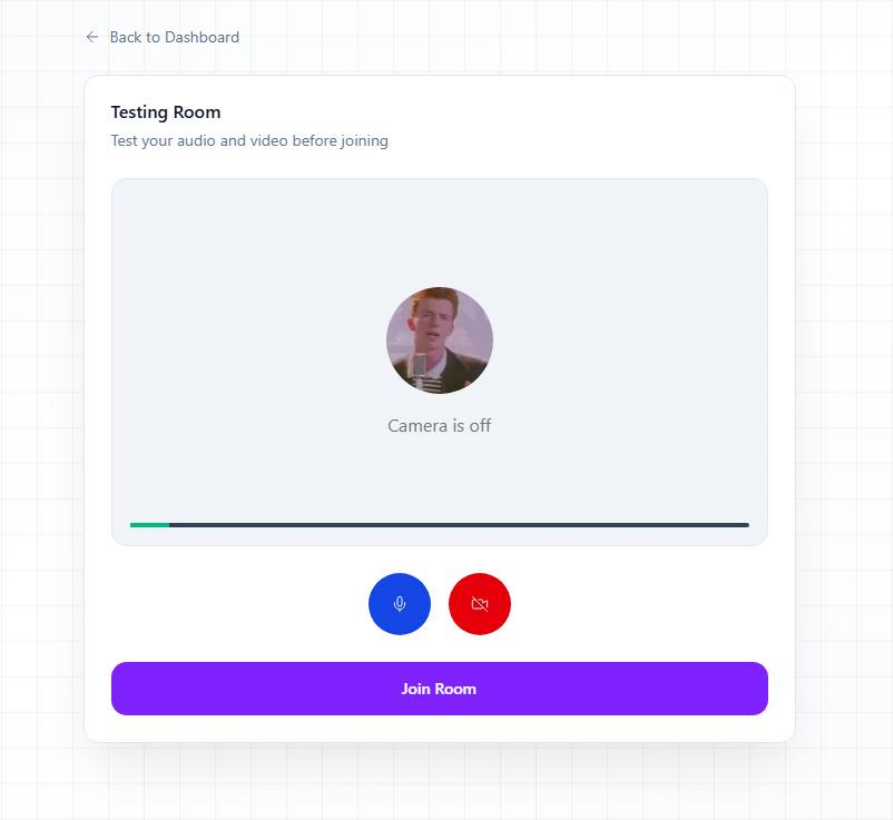

Figure 3-6. Room Joining UI

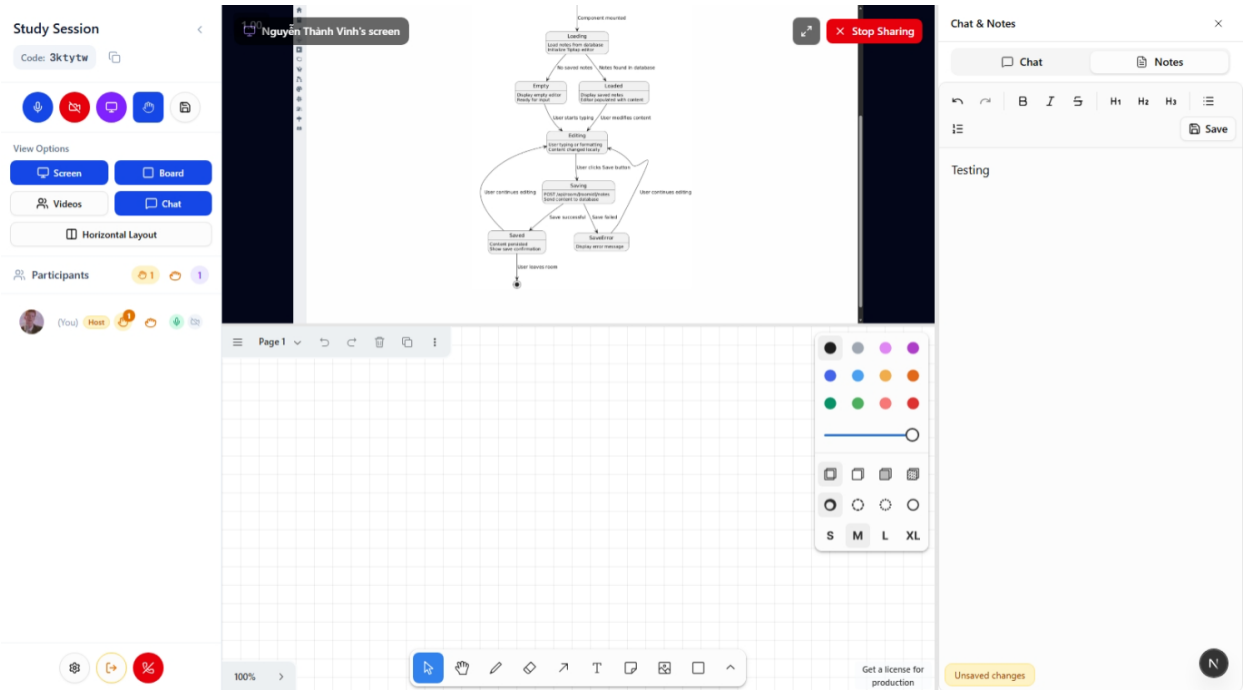

Figure 3-7. Room Meeting UI

Before joining, users can test their microphone and camera, toggle media devices on/off, enter room password if required (non-host only), and then join the room. Once in the room, users can toggle microphone on/off, toggle camera on/off, share their screen with other participants, toggle whiteboard visibility, switch between horizontal and vertical layout for screen share and whiteboard, view and interact with the participants list (see who's speaking, mute status, video status, role badges), raise/lower hand to get attention from host, send chat messages to all participants, take personal notes using the rich-text editor, resize panels (participants sidebar, chat/notes panel) with sizes persisted across sessions, collapse/expand the participants sidebar, show/hide chat and notes panel, manage room settings as host/co-host (permissions, password, co-hosts, kick participants), and leave the room (returns to dashboard). The floating dock auto-hides after inactivity and reappears on hover or interaction.

CONCLUSION

This capstone project successfully developed Stoom, a comprehensive "Study Together Platform" that addresses the fragmentation problem in online collaborative learning by integrating multiple communication and collaboration tools into a unified interface. The project demonstrates the successful application of modern web technologies to create a scalable, real-time collaborative platform with intelligent features.

1. Obtained Results / Achievements

The project has achieved significant milestones:

- **System Completeness**: Successfully integrated Video Conferencing, Collaborative Whiteboard, and Personal Notes into a single interface, eliminating the need to switch between multiple applications during study sessions.
- **Real-time Performance**: Implemented low-latency communication using LiveKit (SFU architecture) for media streaming and LiveKit DataChannel for whiteboard synchronization.
- **Collaborative Whiteboard**: Integrated tldraw library for a full-featured collaborative drawing experience with pen, highlighter, eraser, shapes, and text tools.
- **Personal Notes**: Integrated Tiptap rich-text editor for personal note-taking with support for headings, lists, bold, italic, and strikethrough formatting. Notes can be saved to the database and persist across sessions.
- **Permission System:** Implemented granular permission controls allowing hosts to manage whiteboard and notes access (open, restricted, disabled) and grant/revoke access to specific participants.
- **Room Security**: Implemented room password protection with bcrypt hashing. Hosts can set, change, or remove passwords during active sessions. Password validation is performed during join with host exemption.
- **Role Management**: Implemented a three-tier role system (Host, Co-Host, Participant) with appropriate permission levels. Hosts can grant/revoke co-host privileges. Co-hosts can manage permissions, kick participants, and end meetings.
- **Participant Management**: Implemented hand raise feature for participants to get attention, with queue management for hosts/co-hosts. Added kick functionality to remove disruptive participants with confirmation dialogs.

2. Limitations and Future Development

Limitations

- Dependency Risks: Heavy reliance on third-party services (LiveKit, Clerk, Google Gemini) creates potential single points of failure if service outages occur.
- AI Latency: Summary generation for long sessions may experience delays depending on Gemini API response speed, affecting immediate availability of insights.
- Mobile Experience: Complex interactions like whiteboard drawing are optimized for Desktop/Tablet and may be difficult to use on small smartphone screens.
- Offline Mode: The application requires an active internet connection for all features; no offline support is currently implemented.

Future Development / Orientation

- Mobile App: Develop a native mobile version (React Native/Flutter) to improve the drawing experience on phones with better touch input handling.
- Advanced AI Features: Implement "Quiz Generation" (AI creates quizzes from transcripts) and "Semantic Search" (search for concepts within recorded videos) to transform the platform into a comprehensive learning management system.
- Scheduling Integration: Integrate with Google Calendar and Microsoft Outlook for streamlined study session planning.
- Pro Features: Implement cloud storage management (AWS S3) and premium plans for extended meeting durations, higher quality recordings, and priority AI processing.

The Stoom platform successfully demonstrates the integration of modern web technologies to address real-world challenges in online collaborative learning. While current limitations exist, the foundation has been established for continued development and enhancement.

REFERENCE DOCUMENTS

[1] *Vercel. (n.d.).* ***Next.js Documentation****. Next.js. Retrieved from https://nextjs.org/docs*

[2] *LiveKit. (n.d.).* ***LiveKit Documentation - Real-time video and audio infrastructure****. LiveKit Docs. Retrieved from https://docs.livekit.io/*

[3] *Clerk. (n.d.).* ***Authentication & User Management for Next.js****. Clerk Documentation. Retrieved from https://clerk.com/docs*

[4] *tldraw. (n.d.).* ***tldraw: A library for creating infinite canvas experiences****. tldraw.dev. Retrieved from https://tldraw.dev/*

[5] *Y.js. (n.d.).* ***Shared data types for building collaborative software****. Y.js Docs. Retrieved from https://docs.yjs.dev/*

[6] *Prisma. (n.d.).* ***Prisma: Next-generation Node.js and TypeScript ORM****. Prisma Documentation. Retrieved from https://www.prisma.io/docs/*

[7] *MongoDB. (n.d.).* ***MongoDB Atlas: The Developer Data Platform****. MongoDB Documentation. Retrieved from https://www.mongodb.com/docs/atlas/*

[8] *MDN Web Docs. (n.d.).* ***WebRTC API****. Mozilla Developer Network. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API*

[9] *Google. (n.d.).* ***Gemini API Overview****. Google AI for Developers. Retrieved from https://ai.google.dev/docs*